A PROJECT REPORT ON

SMART ATTENDANCE MANAGEMENT SYSTEM

Streamlining and automating student attendance tracking using Java, Servlets, JSP and MySQL



Submitted in partial fulfilment of the requirements for the award of the degree of

[BACHELOR OF TECHNOLOGY / BCA / MCA] in [COMPUTER SCIENCE & ENGINEERING]

Submitted by

[Student Name]
Roll No.: [Roll Number]
Enrolment No.: [Enrolment Number]

Under the guidance of

[Guide Name]
[Designation]
Department of [Department]

[College / University Name]
[City, State]

Academic Session 2026 - 2027

CERTIFICATE

This is to certify that the project report entitled **"Smart Attendance Management System"** submitted by **[Student Name]**, Roll No. **[Roll Number]**, in partial fulfilment of the requirements for the award of the degree of **[Degree Name]** in **[Department]** of **[College / University Name]**, is a record of the student's own work carried out under my supervision and guidance.

The matter embodied in this report has not been submitted to any other university or institute for the award of any degree or diploma. The project has been found satisfactory and is approved for submission.

[Guide Name]	[Head of Department]
Project Guide	Head of Department
Department of [Department]	Department of [Department]

Date: _____ Place: _____

DECLARATION

I, **[Student Name]**, Roll No. **[Roll Number]**, hereby declare that the project report titled **"Smart Attendance Management System"** is an original work carried out by me under the guidance of **[Guide Name]**. The design, source code, database, diagrams and screenshots presented in this report were prepared by me for the partial fulfilment of the degree of **[Degree Name]**.

The material taken from books, journals, specifications and websites has been acknowledged in the References section. This work has not been submitted, in whole or in part, to any other university or institution for the award of any degree.

Signature: _____
[Student Name]
Roll No.: [Roll Number]
Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, **[Guide Name]**, for the valuable guidance, constant encouragement and constructive criticism that shaped this project from the first idea to the final report.

I am thankful to the Head of the Department, **[Head of Department]**, and to all the faculty members of the Department of **[Department]** for providing the laboratory facilities and a supportive environment. I also thank my classmates who took part in testing the application and whose suggestions made the screens simpler and the workflow faster.

Finally, I am deeply grateful to my family for their patience and support throughout the development of this project.

[Student Name]

ABSTRACT

Taking attendance is one of the most repetitive tasks in every school and college. In most institutions it is still done on paper registers: a teacher calls out names, marks each student by hand, and the registers are later added up by hand to find students who fall below the minimum percentage. This process wastes lecture time, produces calculation errors, allows proxy and back-dated entries without any trace, and tells students about a shortage only when it is already too late to repair it.

This project presents the **Smart Attendance Management System (SAMS)**, a complete web application that replaces the paper register. It is written in **Java** using **Servlets, JSP, JSTL and JDBC**, follows the **MVC** and **DAO** design patterns, stores its data in a **MySQL** database and runs on **Apache Tomcat 9**. Three role-based portals are provided. Teachers mark a whole class in a few clicks and can correct earlier days. Students see their subject-wise percentage at any time, receive a shortage warning and are told how many classes they must attend to recover. Administrators manage users, courses and enrolments, monitor a 14-day trend chart, list defaulters and review an audit log.

To automate attendance completely, the system also offers a **REST API protected by API keys**. RFID readers, biometric terminals or mobile applications can send attendance as JSON; keys are generated by the administrator, stored only as SHA-256 hashes, can be revoked at any time and are limited to 120 requests per minute. Security is built in: PBKDF2 password hashing, CSRF tokens on every form, role checks on every URL, prepared statements, a login lock-out, security headers and a complete audit trail.

The system was tested with 41 automated end-to-end test cases covering authentication, authorisation, attendance marking, report export and the API; all of them passed. With a demo database of 2,400 attendance records the main pages respond in about 40 ms. The report documents the requirements, design (use-case, DFD, ER, class and sequence diagrams, flowcharts), implementation, testing and deployment of the system.

Keywords: attendance management, Java, Servlet, JSP, JDBC, MySQL, REST API, API key, MVC, role-based access control.

TABLE OF CONTENTS

1. Introduction..... 10
1.1 Background..... 10
1.2 Problem Statement..... 10
1.3 Objectives of the Project..... 10
1.4 Scope of the Project..... 11
1.5 Motivation..... 11
1.6 Organisation of the Report..... 11
2. Literature Survey and Feasibility..... 12
2.1 Existing Attendance Methods..... 12
2.2 Limitations of the Existing Systems..... 12
2.3 Proposed System..... 12
2.4 Feasibility Study..... 13
3. System Requirement Specification..... 14
3.1 Purpose and Intended Audience..... 14
3.2 User Classes..... 14
3.3 Functional Requirements..... 14
3.4 Non-Functional Requirements..... 15
3.5 Hardware and Software Requirements..... 15
3.6 Use-Case Model..... 16
4. Technologies Used..... 17
4.1 Java Platform..... 17
4.2 Java Servlets..... 17
4.3 JavaServer Pages and JSTL..... 17
4.4 JDBC and the DAO Pattern..... 17
4.5 MySQL / MariaDB..... 17
4.6 Apache Tomcat..... 18
4.7 HTML5, CSS3 and JavaScript..... 18
4.8 Maven and Supporting Tools..... 18
5. System Design..... 19
5.1 Architectural Design..... 19
5.2 Model-View-Controller Mapping..... 19
5.3 Data Flow Diagrams..... 20
5.4 Database Design..... 21
5.5 Class Design..... 23
5.6 Sequence Diagrams..... 23
5.7 Flowcharts..... 24
5.8 User Interface Design..... 27
5.9 Security Design..... 27
6. Implementation..... 29
6.1 Development Environment..... 29
6.2 Project Structure..... 29
6.3 Configuration and Connection Pool..... 29
6.4 Password Hashing..... 30
6.5 Filters..... 31
6.6 Data Access Layer..... 32
6.7 Attendance Percentage Logic..... 33
6.8 Servlets (Controllers)..... 33
6.9 JSP Views..... 34
6.10 REST API and API Keys..... 34
6.11 Audit Log..... 34
6.12 Front-End Assets..... 35
7. Testing..... 36
7.1 Testing Strategy..... 36
7.2 Test Environment..... 36
7.3 Test Cases and Results..... 36

7.4 Defects Found and Fixed....37
7.5 Performance Measurement....37
8. User Manual with Screenshots....39
8.1 Login Page....39
8.2 Administrator Portal....40
8.3 Teacher Portal....43
8.4 Student Portal....45
9. Deployment Guide....49
9.1 Overview....49
9.2 Step-by-Step Installation....49
9.3 Configuration....49
9.4 Production Recommendations....49
9.5 Troubleshooting....50
10. Results and Discussion....51
10.1 Objectives Versus Achievement....51
10.2 Analysis of the Demo Data....51
10.3 Benefits Observed....51
10.4 Comparison with the Manual Process....52
11. Limitations and Future Scope....53
11.1 Limitations....53
11.2 Future Enhancements....53
12. Conclusion....54
References....55
Appendix A: Database Script (schema.sql)....56

LIST OF FIGURES

Figure 2.1: Project schedule (Gantt chart)..... 13
Figure 3.1: Use-case diagram of SAMS..... 16
Figure 5.1: Three-tier architecture of SAMS..... 19
Figure 5.2: Context-level DFD (Level 0)..... 20
Figure 5.3: Level-1 DFD of SAMS..... 20
Figure 5.4: Entity-relationship diagram of sams_db..... 21
Figure 5.5: Simplified class diagram..... 23
Figure 5.6: Sequence diagram: login..... 23
Figure 5.7: Sequence diagram: taking attendance..... 24
Figure 5.8: Sequence diagram: marking via REST API..... 24
Figure 5.9: Flowchart: authentication..... 25
Figure 5.10: Flowchart: marking attendance..... 25
Figure 5.11: Flowchart: API request processing..... 26
Figure 5.12: Flowchart: student views attendance..... 27
Figure 5.13: Layered security design..... 28
Figure 7.1: Automated assertions per module..... 37
Figure 8.1: Login page..... 39
Figure 8.2: Login failure message..... 39
Figure 8.3: Admin dashboard..... 40
Figure 8.4: User management..... 41
Figure 8.5: Course management..... 41
Figure 8.6: Enrolment of students..... 42
Figure 8.7: API key management..... 42
Figure 8.8: Reports and audit log..... 43
Figure 8.9: Teacher dashboard..... 43
Figure 8.10: Taking attendance..... 44
Figure 8.11: Course report..... 45
Figure 8.12: Student dashboard..... 45
Figure 8.13: Date-wise history..... 46
Figure 8.14: Change password page..... 46
Figure 8.15: Student dashboard on a phone (390 px wide)..... 47
Figure 8.16: Access-denied page..... 47
Figure 10.1: Distribution of attendance percentages..... 51

LIST OF TABLES

Table 1.1: Chapters of this report....11
Table 2.1: Comparison of manual, spreadsheet and proposed system....12
Table 3.1: User classes and their privileges....14
Table 3.2: Functional requirements....14
Table 3.3: Non-functional requirements....15
Table 3.4: Minimum hardware requirements....15
Table 3.5: Software requirements....15
Table 3.6: Description of the main use case: Take attendance....16
Table 4.1: Technology summary....18
Table 5.1: MVC mapping....19
Table 5.2: Table users....21
Table 5.3: Tables students, courses and enrollments....22
Table 5.4: Tables attendance, api_keys and audit_log....22
Table 5.5: Threats and countermeasures....28
Table 6.1: Code size by package....29
Table 7.1: Test environment....36
Table 7.2: Executed test cases (all passed)....36
Table 7.3: Defects found during testing....37
Table 7.4: Measured response time....38
Table 9.1: Configuration keys (db.properties or conf/sams.properties)....49
Table 9.2: Common problems....50
Table 10.1: Achievement of the objectives....51
Table 10.2: Manual process versus SAMS....52

CHAPTER 1

INTRODUCTION

1.1 Background

Attendance is the most basic measure of a student's participation in a course. Most universities and colleges in India and elsewhere require a minimum attendance, commonly 75 percent, before a student is allowed to appear in the examination. Because of this rule, the attendance record is not just a statistic; it directly decides whether a student can continue in a semester. It must therefore be **accurate, tamper-evident and available to the student on time**.

In spite of this importance, attendance in many institutions is still recorded in paper registers or in loosely maintained spreadsheets. A typical class of sixty students takes three to five minutes to be marked, which across five lectures a day and two hundred working days adds up to many hours of lost teaching time. At the end of the month a clerk or the teacher counts present marks by hand, and mistakes are common.

Modern web technology makes it possible to remove these problems at almost no cost. A browser on any computer or phone is enough to take attendance, and the data can be stored in a relational database where percentages are calculated instantly and exactly. Java, with its Servlet and JSP technology, is a mature, secure and widely taught platform for building such enterprise web applications, and it runs on any computer that has a Java Runtime Environment.

1.2 Problem Statement

The manual attendance process suffers from the following specific problems:

- **Time loss:** calling names and signing registers consumes lecture time every day.
- **Human error:** percentages calculated by hand are often wrong; entries can be skipped or duplicated.
- **No accountability:** a register can be changed later without any record of who changed it or when.
- **Late feedback:** students discover a shortage only at the end of the term, when it cannot be repaired.
- **Poor reporting:** finding all defaulters of a department requires adding several registers together.
- **No integration:** manual systems cannot receive data from RFID cards, biometric machines or mobile apps.

The problem to be solved is therefore: **design and build an efficient and accurate web based system that streamlines and automates student attendance tracking, serves teachers, students and administrators, protects the data against misuse and can accept attendance from external devices.**

1.3 Objectives of the Project

1. To develop a Java web application using Servlets, JSP and JDBC with a MySQL back-end.
2. To let a teacher mark attendance of a complete class in under a minute and correct earlier days safely.
3. To calculate attendance percentage automatically and warn students whose attendance is below the threshold.
4. To give the administrator tools for managing users, courses, enrolments, reports and API keys.

5. To expose a REST API, secured by API keys, so that devices and other software can automate attendance entry.
6. To protect the system with password hashing, CSRF protection, role-based access control, rate limiting and an audit log.
7. To provide a clean, responsive user interface that works on desktops, tablets and phones.
8. To test the system systematically and document its design, implementation and deployment.

1.4 Scope of the Project

Within scope: user and course management, enrolment, daily attendance marking and editing, percentage calculation, student and teacher reports, CSV export, defaulter list, dashboard with trend chart, audit log, REST API with API keys, deployment on Tomcat 9 with MySQL or MariaDB.

Outside scope (listed as future work): timetable generation, fee or marks management, face-recognition hardware drivers, SMS or e-mail gateways, and multi-institution (multi-tenant) hosting.

1.5 Motivation

The idea came from a simple observation: the information needed to prevent attendance shortage already exists on the first day of the term, but nobody can see it in time. A system that shows every student his or her percentage after every class turns attendance from a punishment into a self-service tool. At the same time, the system is an excellent opportunity to apply the core ideas of enterprise Java - layered architecture, the DAO pattern, transactions, filters, security and REST - in one realistic, complete project.

1.6 Organisation of the Report

Table 1.1: Chapters of this report

Chapter	Title	Content
1	Introduction	Background, problem, objectives and scope
2	Literature Survey and Feasibility	Existing systems, proposed system, feasibility, schedule
3	System Requirement Specification	Functional and non-functional requirements, use cases
4	Technologies Used	Java, Servlet, JSP, JDBC, MySQL, Tomcat, Maven
5	System Design	Architecture, DFD, ER diagram, class and sequence diagrams, flowcharts, security design
6	Implementation	Module-wise implementation with source code
7	Testing	Test strategy, 41 test cases, defects found, performance
8	User Manual with Screenshots	Every screen of the running system
9	Deployment Guide	Installation, configuration and troubleshooting
10	Results and Discussion	Objectives achieved, data and measurements
11	Limitations and Future Scope	Known limits and planned improvements
12	Conclusion	Summary of the work

CHAPTER 2
LITERATURE SURVEY AND FEASIBILITY

2.1 Existing Attendance Methods

2.1.1 Paper registers

The oldest and still the most common method. It needs no equipment but it is slow, error-prone and impossible to analyse without manual work. Registers can be lost, damaged or altered, and a student can never check his or her record without asking the teacher.

2.1.2 Spreadsheets

Some teachers keep an Excel sheet per class. Calculation is automatic, but the files are scattered on personal computers, are not shared with students, have no access control and no history of changes. Merging the sheets of many teachers for a department report is again a manual job.

2.1.3 Biometric and RFID machines

Fingerprint and RFID terminals identify the student automatically and reduce proxy attendance. However, they usually come with closed software, they record entry to the campus rather than presence in a particular lecture, and integrating them with the college's own records is difficult or expensive.

2.1.4 Commercial ERP and mobile applications

Large ERP packages and mobile attendance apps offer many features, but they are paid, need training, depend on a vendor's cloud, and often cannot be customised to the rules of a particular institution. Small colleges and schools frequently find them too heavy or too costly.

2.2 Limitations of the Existing Systems

- Dependence on paper or on a single teacher's computer - no central, shared, protected data store.
- No real-time visibility for students; shortage found too late.
- No audit trail: records can be changed silently.
- Weak or no integration with devices; every integration is a separate project.
- Licence cost or vendor lock-in for commercial products.

2.3 Proposed System

SAMS is a self-hosted web application that any institution can run on a single ordinary computer. It keeps all attendance in one relational database, shows every role exactly the information it needs, calculates percentages automatically and exposes a documented REST API so that devices can write attendance directly. Because the complete source code is open to the institution, rules such as the 75 percent threshold can be changed in a single configuration file.

Table 2.1: Comparison of manual, spreadsheet and proposed system

Feature	Paper register	Spreadsheet	SAMS (proposed)
Time to mark a class	3 - 5 min	2 - 3 min	< 1 min (one click per student, 'mark all')
Percentage calculation	Manual	Formula	Automatic and instant
Student can see own record	No	No	Yes, 24 x 7

Feature	Paper register	Spreadsheet	SAMS (proposed)
Shortage warning	No	No	Yes, with classes needed to recover
Central database	No	No	Yes (MySQL)
Audit trail of changes	No	No	Yes
Device / API integration	No	No	Yes (API keys)
Access control	Physical	File password	Role based, per URL
Cost	Paper	Office licence	Open-source stack, no licence fee

2.4 Feasibility Study

2.4.1 Technical feasibility

All required technologies - Java, Servlet/JSP, JDBC, MySQL and Tomcat - are free, stable, well documented and run on Windows, Linux and macOS. The application needs no special hardware: any computer that can run a JRE and a database server is sufficient. The prototype was successfully built and tested on such a machine, so the project is technically feasible.

2.4.2 Economic feasibility

There is no licence cost. The only costs are the developer's time and an ordinary computer that most colleges already own. Saving even three minutes per lecture across a whole department repays the effort within a few weeks.

2.4.3 Operational feasibility

The interface was designed so that a teacher needs no training: the attendance page is one table with three buttons per student. Students only read a dashboard. The three roles map exactly to the people who already take part in the manual process, so the system fits the existing way of working.

2.4.4 Schedule feasibility

The project was planned as a sixteen-week activity (Figure 2.1). Work was split into analysis, design, back-end, front-end, API, testing and documentation, with overlap between neighbouring phases.

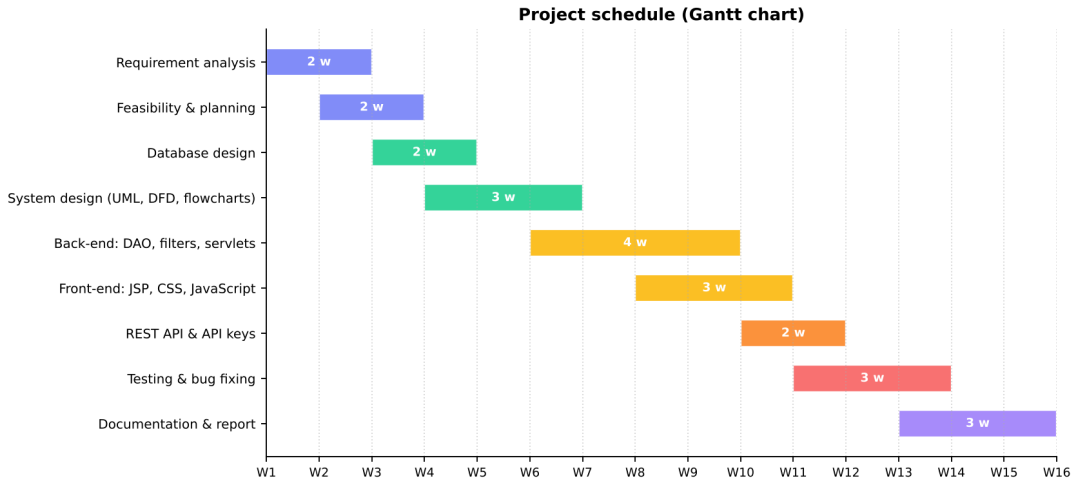


Figure 2.1: Project schedule (Gantt chart)

CHAPTER 3
SYSTEM REQUIREMENT SPECIFICATION

3.1 Purpose and Intended Audience

This chapter lists what the system must do (functional requirements), how well it must do it (non-functional requirements) and on which hardware and software it runs. It follows the structure recommended by IEEE 830 and is intended for the project guide, the examiners and any developer who will maintain the system.

3.2 User Classes

Table 3.1: User classes and their privileges

User	Description	Main privileges
Administrator	Staff member responsible for the system	Create users and courses, assign teachers, enrol students, issue API keys, see all reports and the audit log
Teacher	Faculty member teaching one or more courses	Mark and edit attendance of own courses, view and export own course reports
Student	Learner enrolled in courses	View own percentage, shortage warnings and own history only
External device	RFID reader, biometric terminal or app	Call the REST API with a valid API key to read or mark attendance

3.3 Functional Requirements

Table 3.2: Functional requirements

ID	Requirement
FR-01	The system shall authenticate users by username and password and start a session.
FR-02	The system shall lock a username/IP pair for 5 minutes after 5 consecutive failed logins.
FR-03	The system shall show every user a home page according to the role (admin, teacher, student).
FR-04	The administrator shall create, activate and deactivate users with validated data.
FR-05	The administrator shall create and delete courses and assign a teacher to each course.
FR-06	The administrator shall enrol students in and remove them from a course.
FR-07	A teacher shall mark each student of an own course Present, Late or Absent for a chosen date, with an optional remark.
FR-08	A teacher shall be able to re-open a past date and correct it; the system shall update, not duplicate, the records.
FR-09	The system shall prevent attendance for future dates and for courses the teacher does not teach.
FR-10	The system shall calculate attendance % = (Present + Late) / Total classes x 100 for every student and course.
FR-11	A student shall see subject-wise percentage, a shortage warning below the threshold and the number of consecutive classes needed to recover.
FR-12	A student shall see the date-wise history of each enrolled course and nothing else.
FR-13	A teacher shall view a course report and download it as a CSV file.
FR-14	The administrator shall see a dashboard with totals, a 14-day trend chart and the defaulter list, and shall filter defaulters by percentage.
FR-15	The administrator shall generate, revoke, re-activate and delete API keys; a key is displayed only once.

ID	Requirement
FR-16	The system shall expose a JSON REST API to list courses, read attendance and mark attendance, authenticated by API key.
FR-17	The system shall record logins, data changes and API calls in an audit log visible to the administrator.
FR-18	Every user shall be able to change his or her own password.

3.4 Non-Functional Requirements

Table 3.3: Non-functional requirements

Category	Requirement
Performance	Normal pages shall respond within 100 ms and API calls within 50 ms for a database of several thousand records on an ordinary computer.
Security	Passwords hashed with a slow salted algorithm; CSRF protection for all forms; role checks on all protected URLs; SQL injection and XSS prevented; API keys never stored in clear text.
Reliability	Multi-row writes in one database transaction; UNIQUE key on (course, student, date) guarantees one record per day.
Usability	Clean responsive layout, readable on 360 px wide phones; a teacher can mark a class without training.
Portability	Runs on any OS with Java 11+, Tomcat 9 and MySQL 8 or MariaDB 10.
Maintainability	Layered code (model, DAO, servlet, JSP), small classes, comments, configuration in one properties file.
Auditability	Every important action is logged with user, time and IP address.

3.5 Hardware and Software Requirements

Table 3.4: Minimum hardware requirements

Component	Server	Client
Processor	Dual-core 1.6 GHz	Any modern processor
Memory	2 GB RAM (4 GB recommended)	1 GB
Disk	500 MB free	-
Network	LAN / Wi-Fi	LAN / Wi-Fi
Display	-	Any, from 360 px width

Table 3.5: Software requirements

Item	Version / choice
Operating system	Windows 10/11, Ubuntu 22.04+, macOS (any)
Java	JDK 11 or later (JRE is enough to run)
Application server	Apache Tomcat 9.0.x (Servlet 4.0, JSP 2.3)
Database	MySQL 8.x or MariaDB 10.5+
Build tool	Apache Maven 3.8+ (or an IDE with Maven support)
JDBC driver	MySQL Connector/J 8.x (MariaDB driver in the prebuilt WAR)
Libraries	JSTL 1.2, Servlet API 4.0 (provided by Tomcat)
Browser	Chrome, Firefox, Edge, Safari (current versions)
IDE (optional)	Eclipse, IntelliJ IDEA or NetBeans

3.6 Use-Case Model

Figure 3.1 shows the actors and their use cases. All three human actors share login, logout and change password; the remaining use cases are role specific. The external device is a non-human actor that uses only the API.

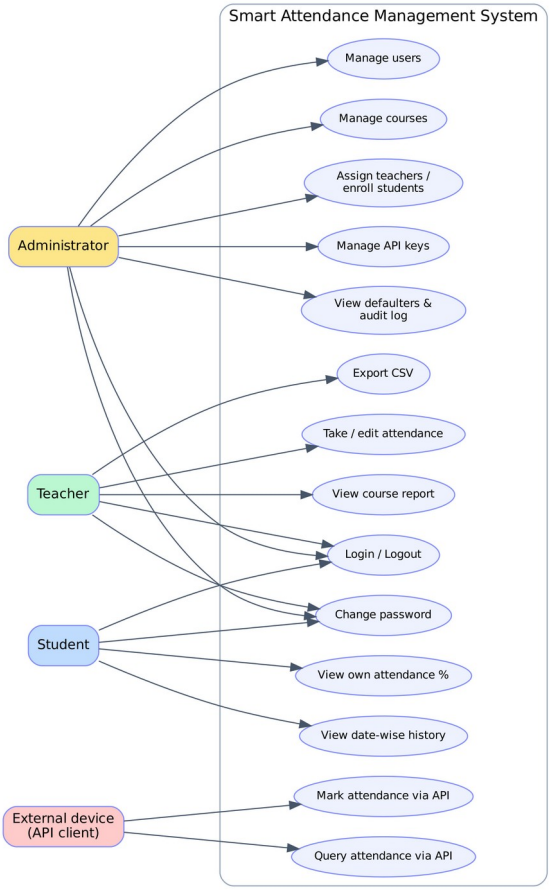


Figure 3.1: Use-case diagram of SAMS

Table 3.6: Description of the main use case: Take attendance

Item	Description
Use case	UC-08 Take / edit attendance
Actor	Teacher
Pre-condition	Teacher is logged in and the course is assigned to him / her; students are enrolled.
Main flow	1. Teacher opens a course. 2. System shows the class list for today (or the chosen date) with existing marks if any. 3. Teacher sets P / L / A per student, optionally uses 'Mark all present'. 4. Teacher presses Save. 5. System validates, saves all rows in one transaction, writes an audit entry and shows a confirmation.
Alternative flow	A1: date in the future -> error message. A2: a student has no status -> error and the form is shown again. A3: course not owned by the teacher -> HTTP 403.
Post-condition	Exactly one record per student, course and date exists in the attendance table.

CHAPTER 4

TECHNOLOGIES USED

4.1 Java Platform

Java is a general-purpose, object-oriented, platform-independent language. Source code is compiled to byte-code that runs on the Java Virtual Machine, so the same WAR file runs unchanged on Windows, Linux and macOS. Strong typing, automatic memory management, a rich standard library (JDBC, java.security, javax.crypto, java.time) and decades of enterprise use make Java a natural choice for a data-centred business application such as attendance management. This project uses Java 11 language features and was built and tested with JDK 21.

4.2 Java Servlets

A servlet is a Java class that runs inside a web container and handles HTTP requests. The container (Tomcat) manages the servlet's life cycle - `init()`, `service()` and `destroy()` - and calls `doGet()` or `doPost()` for each request. Servlets are ideal controllers because they can read request parameters, call business code and decide which view to show. SAMS uses seventeen servlet classes, three filters and one context listener:

- **Servlets** receive requests, call the DAO layer and forward to a JSP (e.g. `TeacherAttendanceServlet`).
- **Filters** run before servlets and apply cross-cutting rules: `SecurityFilter` (CSRF, headers), `AuthFilter` (login and role), `ApiKeyFilter` (API keys).
- **Listener** `AppContextListener` starts the connection pool at start-up and closes it at shutdown.

4.3 JavaServer Pages and JSTL

JSP lets a developer write HTML with dynamic parts. A JSP is translated once into a servlet and then compiled, so it is as fast as hand-written servlet code. The JSP Standard Tag Library (JSTL) provides tags such as `c:forEach`, `c:if`, `c:choose` and `fmt:formatDate`, and the Expression Language (EL) `${...}` reads data from the request or session. In SAMS the JSP files contain **no Java scriptlets** at all (except the small error page); all logic stays in servlets and DAOs, and the output of user data is always escaped with `c:out` to prevent cross-site scripting.

4.4 JDBC and the DAO Pattern

Java Database Connectivity (JDBC) is the standard API for talking to relational databases. SAMS uses only `PreparedStatement`, which sends the SQL text and the parameter values separately and therefore makes SQL injection impossible. Each table group has a Data Access Object (DAO) class that hides all SQL from the rest of the program, as described by Fowler [9]. Multi-step changes use `setAutoCommit(false)`, `commit()` and `rollback()` to behave as one atomic transaction.

4.5 MySQL / MariaDB

MySQL is the most widely used open-source relational database. The schema uses the InnoDB engine, which supports foreign keys and transactions, `ENUM` columns for fixed value sets, `UNIQUE` keys for integrity and a SQL **view** (`v_attendance_summary`) that calculates the attendance percentage so that the formula exists in only one place. The same SQL works on MariaDB, which is the database shipped with the XAMPP package popular in colleges.

4.6 Apache Tomcat

Apache Tomcat 9 is the reference implementation of the Servlet 4.0 and JSP 2.3 specifications. It is distributed as a small download, starts in seconds and deploys a WAR file by simply copying it into the `webapps` folder. Tomcat 9 uses the `javax.servlet` package names, which is why this project targets Tomcat 9 and not Tomcat 10+ (which switched to `jakarta.servlet`).

4.7 HTML5, CSS3 and JavaScript

The user interface is plain HTML5 and CSS3 with a small vanilla-JavaScript file (78 lines) for conveniences such as 'mark all present', live counters, table filtering and confirmation dialogs. No framework is needed, which keeps pages light and the code easy to read. Because a Content-Security-Policy header forbids inline scripts and styles, all behaviour is attached through `data-*` attributes and CSS utility classes.

4.8 Maven and Supporting Tools

Apache Maven describes the project in `pom.xml` (packaging `war`, dependencies Servlet API, JSP API, JSTL, MySQL Connector/J) and builds the deployable WAR with `mvn clean package`. Graphviz and Matplotlib were used for the diagrams of this report, and a headless Chromium browser was used to capture the screenshots.

Table 4.1: Technology summary

Layer	Technology	Role in SAMS
Presentation	JSP 2.3, JSTL 1.2, HTML5, CSS3, JavaScript	Role-based pages, forms, tables, charts
Controller	Servlets 4.0, Filters, Listener	Request handling, security, start-up
Business / data access	Java 11, JDBC, DAO classes, DBUtil pool	Rules, SQL, transactions
Database	MySQL 8 / MariaDB 10	7 tables, 1 view, constraints
Server	Apache Tomcat 9	Hosts the WAR
Build	Maven 3.8+	Dependencies and WAR packaging

CHAPTER 5
SYSTEM DESIGN

5.1 Architectural Design

SAMS is a classic three-tier web application. The **client tier** is a web browser (or a device that speaks HTTP). The **application tier** runs in Tomcat and is itself layered: filters, servlets (controllers), JSP (views), DAO classes and the connection pool. The **data tier** is the MySQL database. Requests from browsers pass through `SecurityFilter` and `AuthFilter`; requests from devices pass through `ApiKeyFilter` to `ApiServlet`.

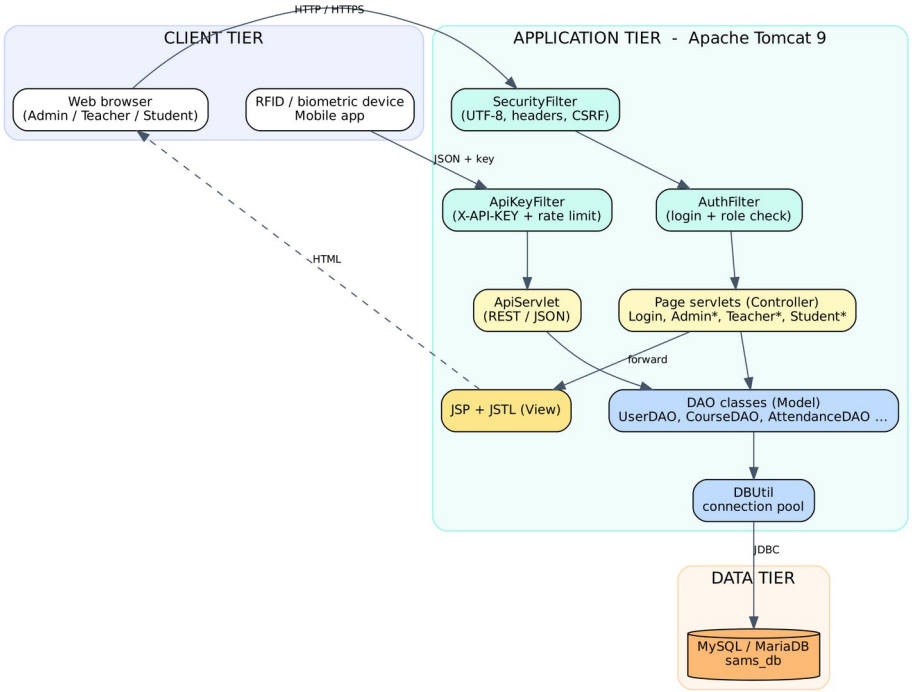


Figure 5.1: Three-tier architecture of SAMS

5.2 Model-View-Controller Mapping

Table 5.1: MVC mapping

MVC role	Implementation	Examples
Model	Plain Java beans + DAO classes	`User`, `Course`, `AttendanceSummary`, `AttendanceDAO`
View	JSP pages with JSTL / EL	`teacher/attendance.jsp`, `student/dashboard.jsp`
Controller	HTTP servlets	`TeacherAttendanceServlet`, `LoginServlet`
Cross-cutting	Filters, listener	`AuthFilter`, `SecurityFilter`, `AppContextListener`

JSP files are stored under `WEB-INF/views`, which Tomcat never serves directly. A browser can therefore reach a page only through a servlet, so no view can be opened without passing the filters.

5.3 Data Flow Diagrams

The context-level DFD (Figure 5.2) shows SAMS as a single process exchanging data with four external entities. Level 1 (Figure 5.3) opens the process into five sub-processes and four data stores.

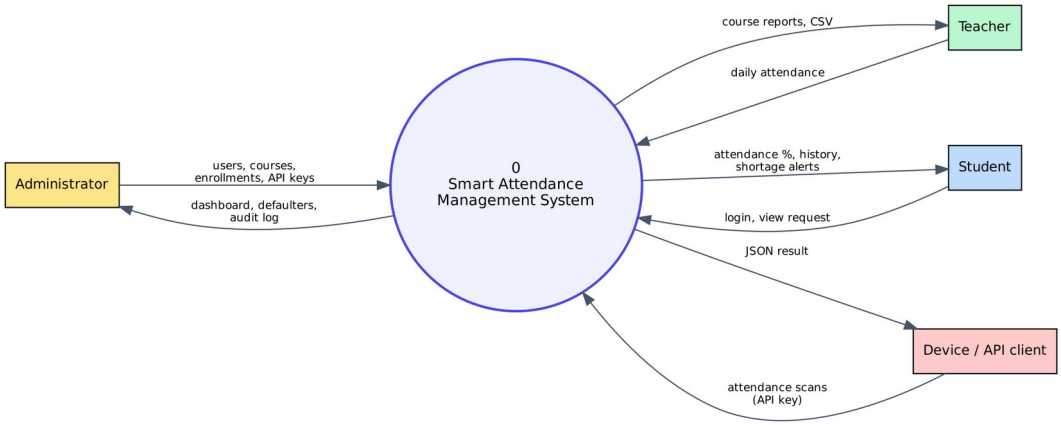


Figure 5.2: Context-level DFD (Level 0)

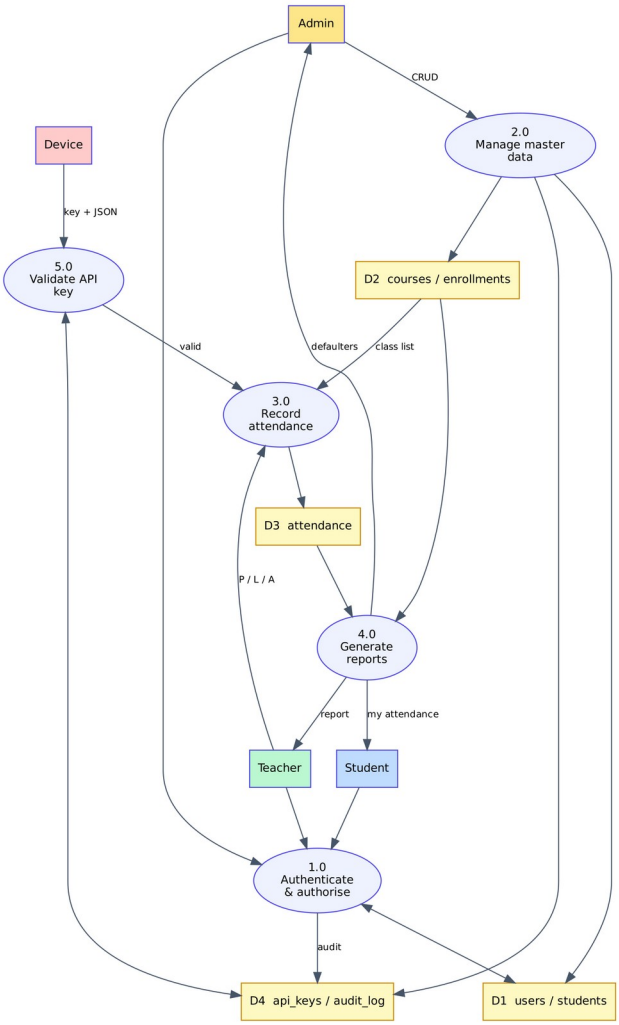


Figure 5.3: Level-1 DFD of SAMS

5.4 Database Design

5.4.1 Entity-relationship diagram

The database `sams_db` contains seven tables. `users` is the central entity; `students` extends it for the student role; `courses` refer to the teacher; `enrollments` implement the many-to-many relation between students and courses; `attendance` holds one row per student, per course and per day; `api_keys` and `audit_log` support integration and accountability.

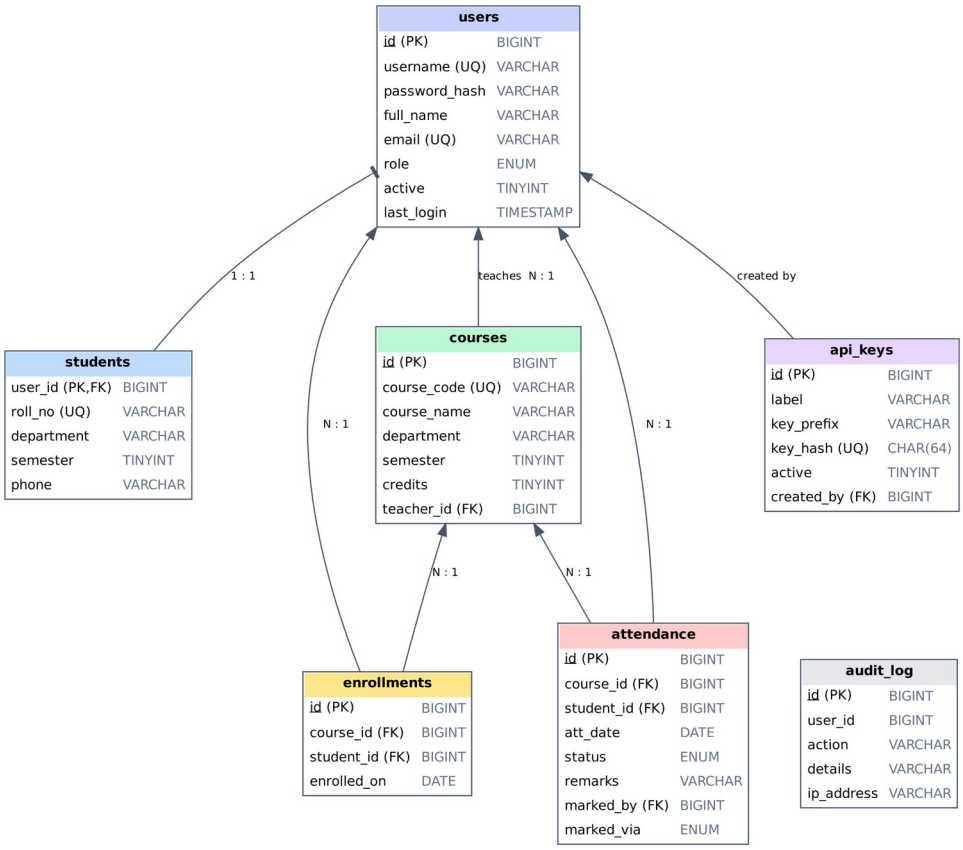


Figure 5.4: Entity-relationship diagram of sams_db

5.4.2 Data dictionary

Table 5.2: Table users

Column	Type	Constraint	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique user id
username	VARCHAR(50)	UNIQUE, NOT NULL	Login name
password_hash	VARCHAR(255)	NOT NULL	PBKDF2 hash string
full_name	VARCHAR(100)	NOT NULL	Display name
email	VARCHAR(120)	UNIQUE, NOT NULL	E-mail address
role	ENUM	NOT NULL	ADMIN / TEACHER / STUDENT
active	TINYINT(1)	DEFAULT 1	0 = account disabled
created_at, last_login	TIMESTAMP		Audit columns

Table 5.3: Tables students, courses and enrollments

Table.Column	Type	Constraint	Description
students.user_id	BIGINT	PK, FK -> users	Student account
students.roll_no	VARCHAR(30)	UNIQUE	Roll number
students.department / semester / phone	VARCHAR / TINYINT	NOT NULL	Profile data
courses.id	BIGINT	PK	Course id
courses.course_code	VARCHAR(20)	UNIQUE	e.g. CS501
courses.course_name / department	VARCHAR	NOT NULL	Course data
courses.semester / credits	TINYINT	NOT NULL	Course data
courses.teacher_id	BIGINT	FK -> users, SET NULL	Teacher in charge
enrollments.course_id, student_id	BIGINT	FK, UNIQUE pair	Who studies what

Table 5.4: Tables attendance, api_keys and audit_log

Table.Column	Type	Constraint	Description
attendance.id	BIGINT	PK	Record id
attendance.course_id, student_id	BIGINT	FK, CASCADE	Whose attendance, in which course
attendance.att_date	DATE	UNIQUE with course+student	Class date
attendance.status	ENUM	NOT NULL	PRESENT / ABSENT / LATE
attendance.remarks	VARCHAR(200)	NULL	Optional note
attendance.marked_by	BIGINT	FK, SET NULL	Teacher (NULL for API)
attendance.marked_via	ENUM	DEFAULT WEB	WEB or API
api_keys.key_hash	CHAR(64)	UNIQUE	SHA-256 of the key
api_keys.key_prefix	VARCHAR(12)		First 12 characters for display
api_keys.active	TINYINT(1)	DEFAULT 1	Revoked keys are rejected
audit_log.action / details / ip_address	VARCHAR		What happened, where from

5.4.3 Normalisation

All tables are in Third Normal Form. Repeating groups (a student's courses) are moved to `enrollments`; every non-key column depends only on the key of its own table; the teacher's name is not copied into `courses` but obtained with a join. The only derived data, the attendance percentage, is not stored but calculated by the view `v_attendance_summary`, so it can never become inconsistent with the records.

5.5 Class Design

Figure 5.5 shows the main classes. All page servlets extend `BaseServlet`, which turns checked exceptions into a common error page and provides helpers for forwarding, redirecting and audit logging. Each DAO obtains its connection from `DBUtil`.

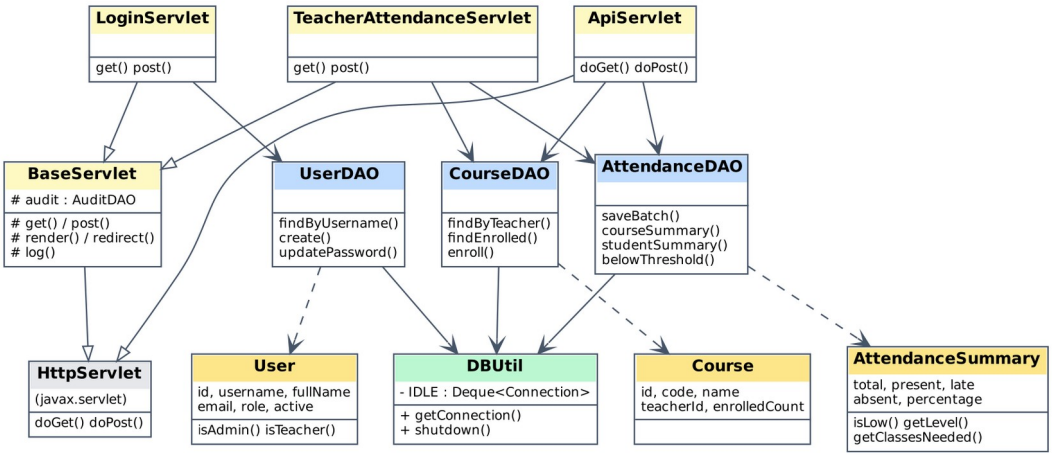


Figure 5.5: Simplified class diagram

5.6 Sequence Diagrams

The following sequence diagrams show the order of calls for the three most important scenarios: login, taking attendance and marking attendance by a device through the API.

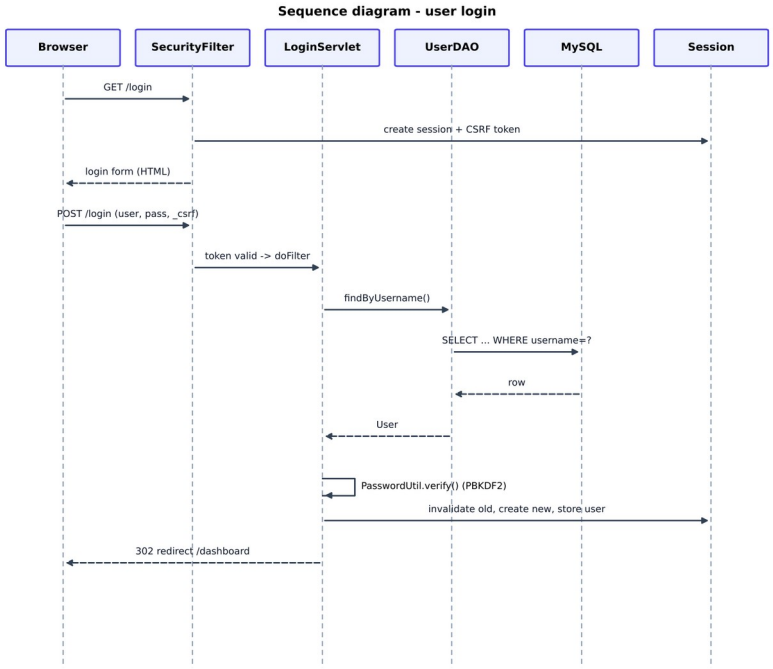


Figure 5.6: Sequence diagram: login

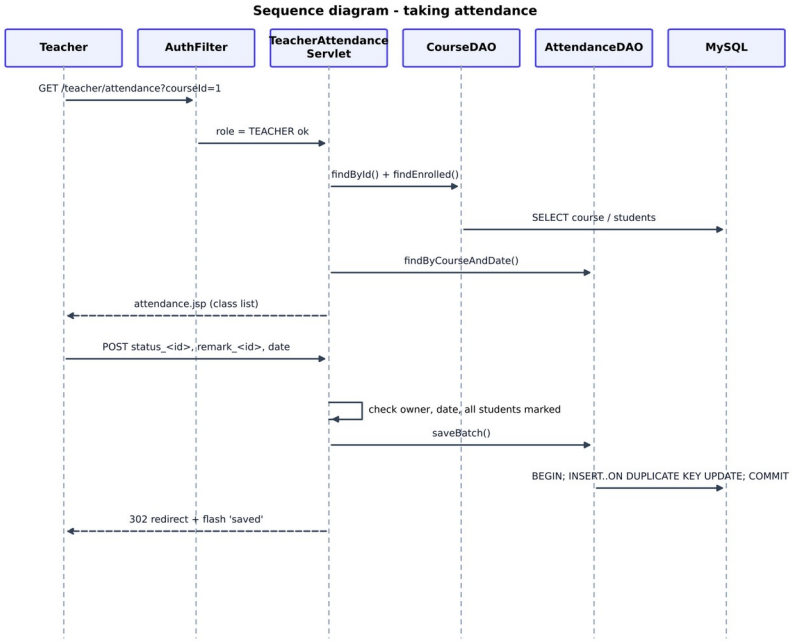


Figure 5.7: Sequence diagram: taking attendance

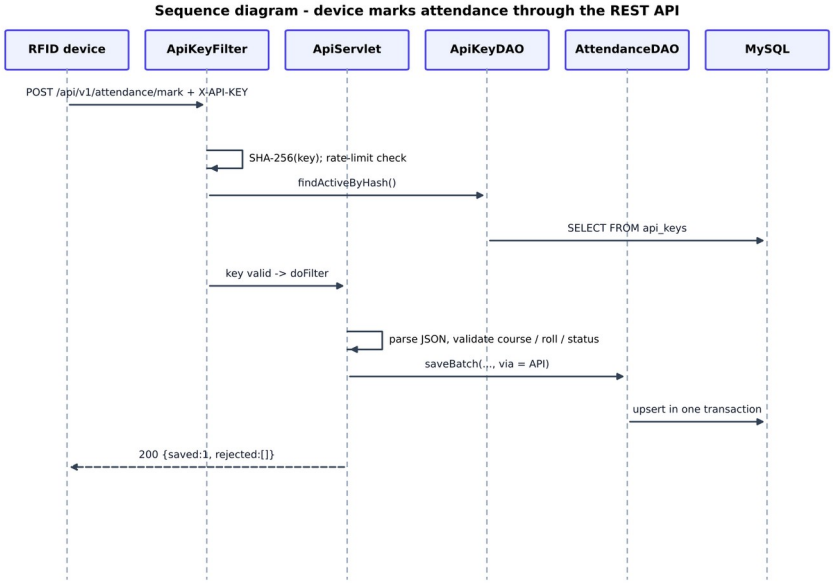


Figure 5.8: Sequence diagram: marking via REST API

5.7 Flowcharts

The flowcharts below describe the decision logic of the four most important processes. Rounded boxes are start/end, rectangles are processing, parallelograms are input/output and diamonds are decisions.

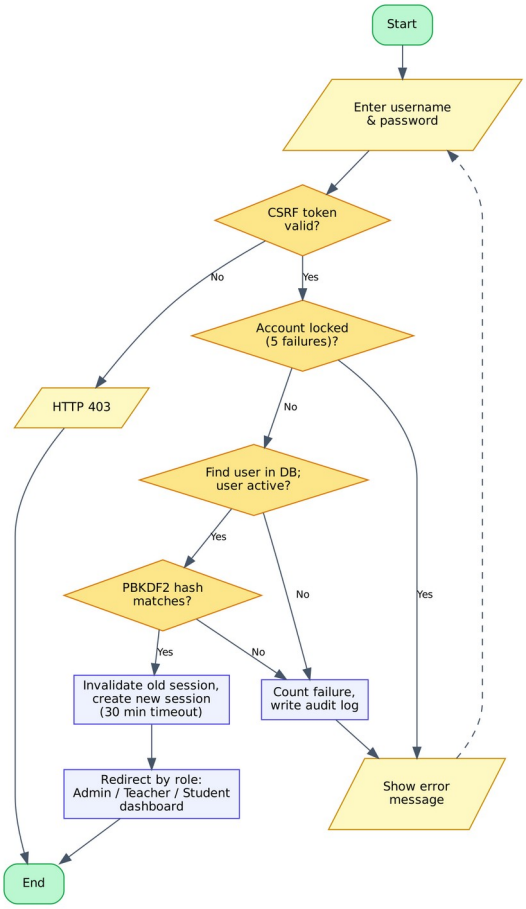


Figure 5.9: Flowchart: authentication

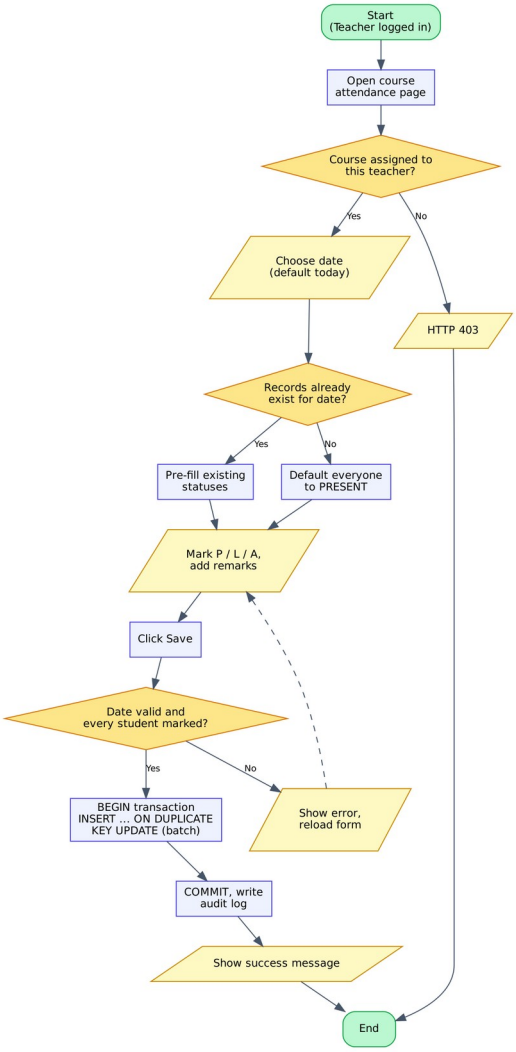


Figure 5.10: Flowchart: marking attendance

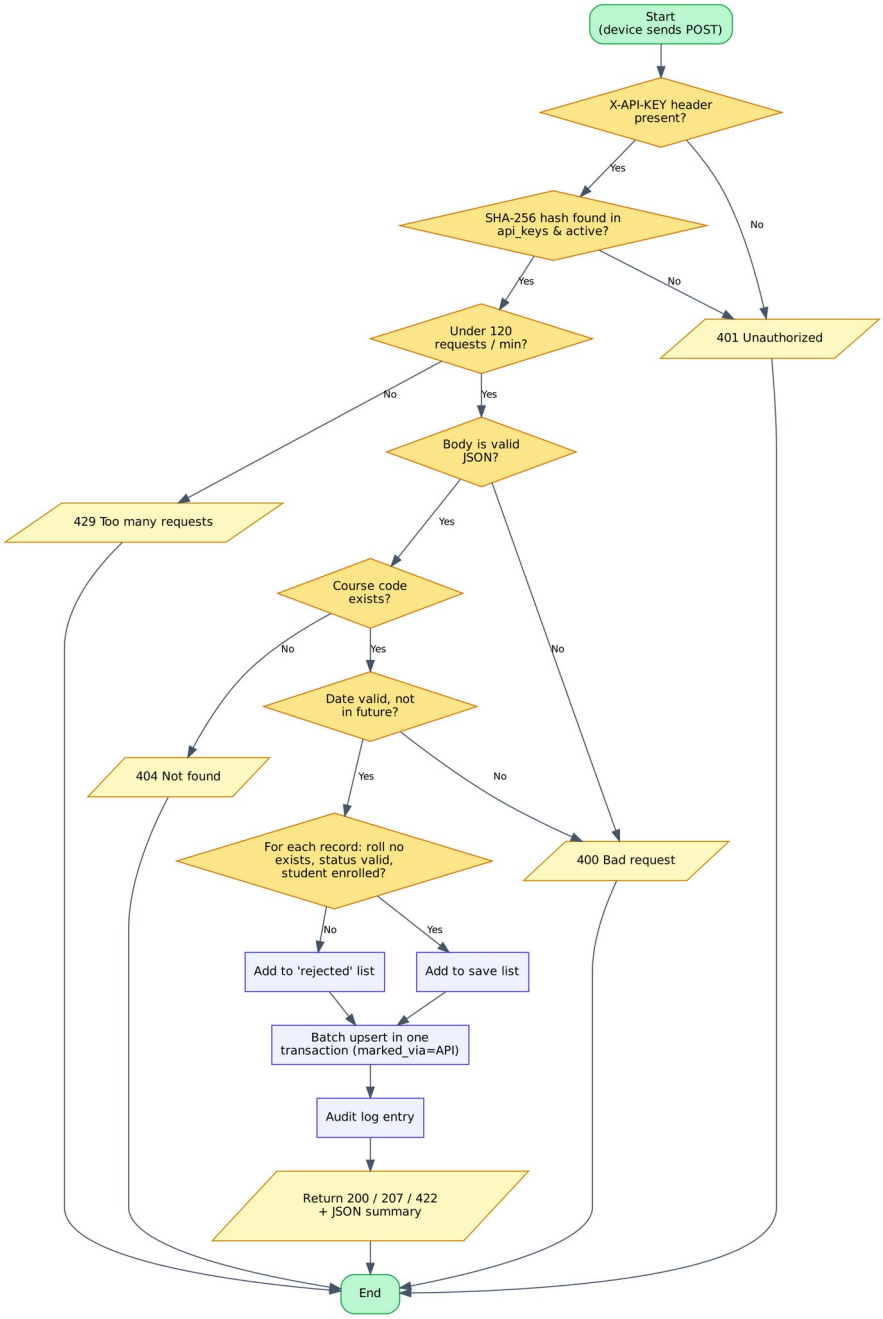


Figure 5.11: Flowchart: API request processing

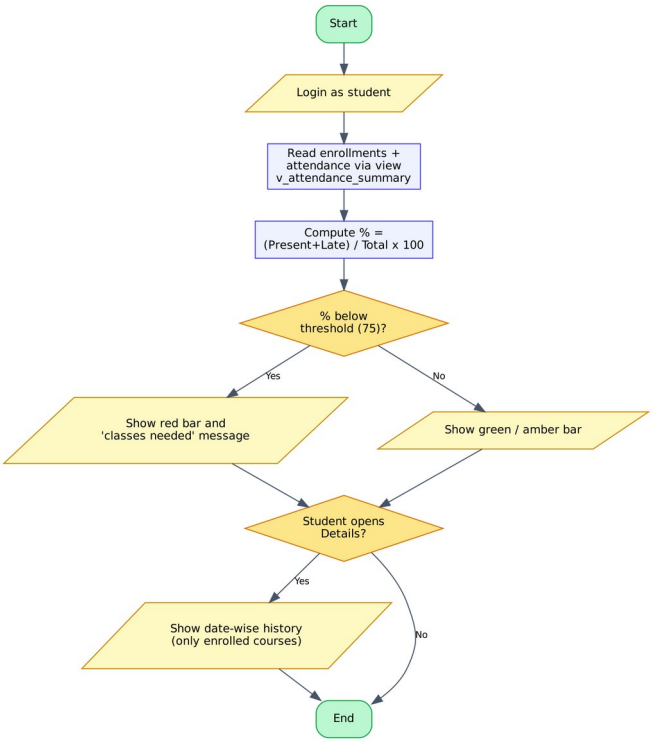


Figure 5.12: Flowchart: student views attendance

5.8 User Interface Design

The interface follows five principles: (1) **role-based navigation** - each user sees only the menu entries he or she may use; (2) **one task per page**; (3) **immediate feedback** through colour-coded badges (green good, amber warning, red shortage) and flash messages; (4) **responsive layout** based on flexible boxes so that the pages remain usable on a phone; (5) **accessibility** through real `label` elements, sufficient contrast and keyboard-operable controls. The attendance page uses a segmented P / L / A control per student because it is faster than a drop-down and shows the state at a glance.

5.9 Security Design

Security is designed as six layers (Figure 5.13). An attacker has to defeat every layer, and each layer is implemented in a small, testable class.

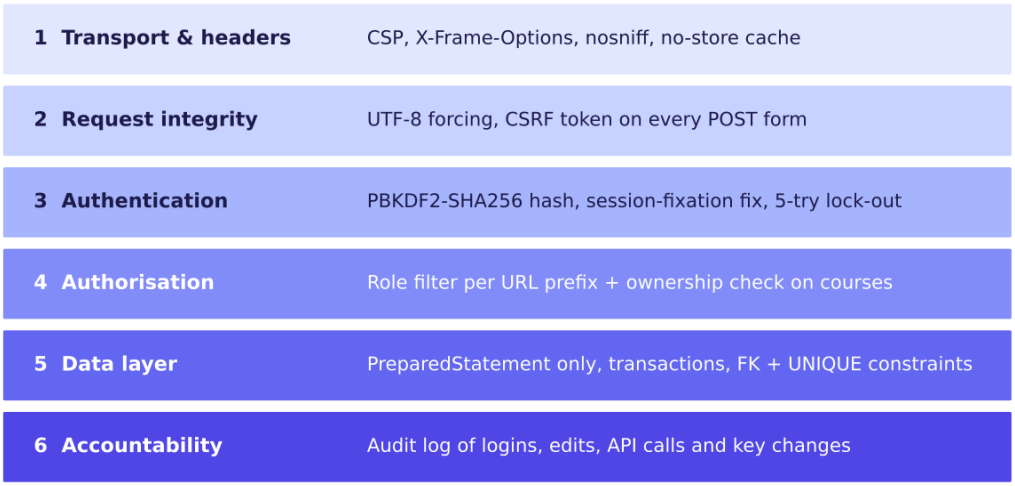


Figure 5.13: Layered security design

Table 5.5: Threats and countermeasures

Threat	Countermeasure in SAMS
Stolen database / password theft	PBKDF2-HMAC-SHA256, 65,000 iterations, 16-byte random salt per password
Brute-force password guessing	Lock-out of username + IP for 5 minutes after 5 failures; failures are audited
Session fixation	Old session invalidated and a new one created on login; HttpOnly cookie; 30 minute timeout
Cross-site request forgery	Per-session random token required in every POST form (`SecurityFilter`)
SQL injection	Only `PreparedStatement` with bound parameters
Cross-site scripting	All output escaped with `c:out`; Content-Security-Policy forbids inline script
Clickjacking / MIME sniffing	`X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`
Privilege escalation	Role check per URL prefix in `AuthFilter`; teacher may open only own courses; student only own history
Leaked API key	Only SHA-256 hash stored; key shown once; revocable; rate limit of 120 requests/min
Spreadsheet formula injection in CSV	Cells beginning with `=`, `+`, `-` or `@` are prefixed with an apostrophe
Back button after logout	`Cache-Control: no-store` on all dynamic pages

CHAPTER 6
IMPLEMENTATION

6.1 Development Environment

The system was developed with JDK 21 (compiling for Java 11), Apache Tomcat 9 libraries, MariaDB 10.11 / MySQL syntax, JSTL 1.2 and a plain text editor; the Maven project can be opened unchanged in Eclipse, IntelliJ IDEA or NetBeans. The finished project contains 41 Java classes (2,700 lines), 16 JSP files (710 lines), 171 lines of CSS, 78 lines of JavaScript and 160 lines of SQL.

Table 6.1: Code size by package

Package	Files	Lines	Responsibility
com.sams.model	8	282	Plain data classes (beans)
com.sams.dao	6	735	All SQL and transactions
com.sams.servlet	17	978	Controllers and REST API
com.sams.filter	3	190	Security, authentication, API keys
com.sams.util	6	481	Pool, hashing, JSON, helpers, configuration
com.sams.listener	1	34	Start-up / shutdown

6.2 Project Structure

Listing 6.1: Folder layout

```
AttendanceManagementSystem
|-- pom.xml                        Maven build file (WAR)
|-- database/  schema.sql  sample_data.sql
|-- docs/      API.md
|-- tests/     smoke_test.sh
`-- src/main
    |-- java/com/sams/{model,dao,servlet,filter,listener,util}
    |-- resources/db.properties
    `-- webapp/
        |-- WEB-INF/web.xml
        |-- WEB-INF/views/{common,admin,teacher,student}/*.jsp
        |-- css/style.css   js/app.js   images/favicon.svg
```

6.3 Configuration and Connection Pool

`Config` reads `db.properties` from the class path. Values can be overridden by a file `conf/sams.properties` in the Tomcat folder or by `-Dsams.db.url=...` on the command line, so the WAR never needs to be edited when the database password changes.

Opening a database connection is expensive, so `DBUtil` implements a small pool. A semaphore limits the number of connections in use (default 10); idle connections are kept in a deque and validated with `isValid()` before reuse. The caller receives a **dynamic proxy**: calling `close()` on it does not close the physical connection but returns it to the pool, and any transaction left open is rolled back. This lets every DAO method use the familiar try-with-resources pattern.

Listing 6.2: DBUtil.getConnection() and the proxy that returns connections to the pool

```
    public static Connection getConnection() throws SQLException {
        if (!ready) init();
        try {
            if (!permits.tryAcquire(10, TimeUnit.SECONDS)) {
                throw new SQLException("Database connection pool exhausted");
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
```

```
            throw new SQLException("Interrupted while waiting for a connection", e);
        }
        Connection real = null;
        try {
            while ((real = IDLE.pollFirst()) != null) {
                if (real.isValid(2)) break;
                closeQuietly(real);
                real = null;
            }
            if (real == null) real = DriverManager.getConnection(url, user, password);
            return wrap(real);
        } catch (SQLException | RuntimeException e) {
            closeQuietly(real);
            permits.release();
            throw e;
        }
    }

    private static Connection wrap(final Connection real) {
        final AtomicBoolean closed = new AtomicBoolean(false);
        return (Connection) Proxy.newProxyInstance(
                DBUtil.class.getClassLoader(), new Class<?>[]{Connection.class},
                (proxy, method, args) -> {
                    String name = method.getName();
                    if ("close".equals(name)) {
                        if (closed.compareAndSet(false, true)) release(real);
                        return null;
                    }
                    if ("isClosed".equals(name)) return closed.get();
                    try {
                        return method.invoke(real, args);
                    } catch (InvocationTargetException e) {
                        throw e.getCause();
                    }
                });
    }

    private static void release(Connection real) {
        try {
            if (!real.isClosed()) {
                if (!real.getAutoCommit()) {          // roll back anything left open
                    real.rollback();
                    real.setAutoCommit(true);
                }
                IDLE.offerFirst(real);
            }
        } catch (SQLException e) {
            closeQuietly(real);
```

6.4 Password Hashing

`PasswordUtil` uses the PBKDF2 algorithm of the JDK, as specified in RFC 8018 [6]. The stored string has the form `pbkdf2$65000$salt$hash`, so the iteration count can be raised later without breaking old passwords. Verification recomputes the hash and compares it in constant time with `MessageDigest.isEqual`.

Listing 6.3: Password hashing and verification

```
    public static String hash(String password) {
        byte[] salt = new byte[16];
        RANDOM.nextBytes(salt);
        byte[] dk = derive(password.toCharArray(), salt, ITERATIONS);
        Base64.Encoder b64 = Base64.getEncoder();
        return "pbkdf2$" + ITERATIONS + "$" + b64.encodeToString(salt) + "$" +
b64.encodeToString(dk);
    }

    public static boolean verify(String password, String stored) {
        if (password == null || stored == null) return false;
        String[] p = stored.split("\\$");
        if (p.length != 4 || !"pbkdf2".equals(p[0])) return false;
        try {
            int iterations = Integer.parseInt(p[1]);
            byte[] salt = Base64.getDecoder().decode(p[2]);
            byte[] expected = Base64.getDecoder().decode(p[3]);
            byte[] actual = derive(password.toCharArray(), salt, iterations);
```

```
            return MessageDigest.isEqual(expected, actual);   // constant-time comparison
        } catch (IllegalArgumentException e) {
            return false;
        }
    }

    /** Minimum policy: 8+ characters with at least one letter and one digit. */
    public static String validateStrength(String pw) {
        if (pw == null || pw.length() < 8) return "Password must be at least 8 characters
long.";
        boolean letter = false, digit = false;
        for (char c : pw.toCharArray()) {
            if (Character.isLetter(c)) letter = true;
            if (Character.isDigit(c)) digit = true;
        }
        return (letter && digit) ? null : "Password must contain at least one letter and one
digit.";
    }
```

6.5 Filters

SecurityFilter is mapped to `/*`. It forces UTF-8, adds the security headers, creates the CSRF token in the session and rejects any POST whose `_csrf` parameter does not match. **AuthFilter** is mapped to the protected URL prefixes and sends anonymous users to the login page and users of the wrong role to a 403 page. **ApiKeyFilter** protects `/api/v1/*`.

Listing 6.4: Role check in AuthFilter

```
    @Override
    public void doFilter(ServletRequest rq, ServletResponse rs, FilterChain chain) throws
IOException, ServletException {
        HttpServletRequest req = (HttpServletRequest) rq;
        HttpServletResponse resp = (HttpServletResponse) rs;
        HttpSession session = req.getSession(false);
        User user = session == null ? null : (User) session.getAttribute("user");

        if (user == null) {
            resp.sendRedirect(req.getContextPath() + "/login?expired=1");
            return;
        }
        String path = req.getRequestURI().substring(req.getContextPath().length());
        boolean allowed =
            (path.startsWith("/admin/")   && user.isAdmin())   ||
            (path.startsWith("/teacher/") && user.isTeacher()) ||
            (path.startsWith("/student/") && user.isStudent()) ||
            path.startsWith("/account/") || path.equals("/dashboard");
        if (!allowed) {
            resp.sendError(403, "You do not have permission to open this page.");
            return;
        }
        chain.doFilter(rq, rs);
    }
```

Listing 6.5: API key verification and rate limit

```
    @Override
    public void doFilter(ServletRequest rq, ServletResponse rs, FilterChain chain) throws
IOException, ServletException {
        HttpServletRequest req = (HttpServletRequest) rq;
        HttpServletResponse resp = (HttpServletResponse) rs;

        if (req.getRequestURI().endsWith("/api/v1/health")) {       // public heartbeat
            chain.doFilter(rq, rs);
            return;
        }
        String key = req.getHeader("X-API-KEY");
        if (key == null) {
            String auth = req.getHeader("Authorization");
            if (auth != null && auth.regionMatches(true, 0, "Bearer ", 0, 7)) key =
auth.substring(7).trim();
        }
        if (key == null || key.isEmpty()) { error(resp, 401, "Missing API key. Send it in the X-
API-KEY header."); return; }

        try {
```

```
            ApiKey k = dao.findActiveByHash(ApiKeyUtil.sha256(key));
            if (k == null) { error(resp, 401, "Invalid or revoked API key."); return; }
            if (!allow(k.getId())) {
                resp.setHeader("Retry-After", "60");
                error(resp, 429, "Rate limit exceeded (" + LIMIT_PER_MINUTE + " requests per
minute).");
                return;
            }
            dao.touch(k.getId());
            req.setAttribute("apiKey", k);
        } catch (SQLException e) {
            error(resp, 500, "Database error.");
            return;
        }
        chain.doFilter(rq, rs);
    }

    private boolean allow(long keyId) {
        long now = System.currentTimeMillis();
        long[] w = windows.computeIfAbsent(keyId, id -> new long[]{now, 0});
        synchronized (w) {
            if (now - w[0] > 60_000) { w[0] = now; w[1] = 0; }
            return ++w[1] <= LIMIT_PER_MINUTE;
        }
    }
```

6.6 Data Access Layer

Every DAO method opens a connection from the pool, prepares a parameterised statement and maps the result set to model objects. The most important method is `AttendanceDAO.saveBatch()`. It writes a whole class in **one transaction** using `INSERT ... ON DUPLICATE KEY UPDATE`: if the teacher saves the same date again, existing rows are updated instead of duplicated, thanks to the `UNIQUE (course_id, student_id, att_date)` key. If any row fails, `rollback()` leaves the database unchanged.

Listing 6.6: AttendanceDAO.saveBatch()

```
    }

    /**
     * Saves a whole class in one transaction. A second save for the same day
     * UPDATES the rows (UNIQUE key + ON DUPLICATE KEY UPDATE) - so teachers can fix mistakes.
     *
     * @return number of students saved
     */
    public int saveBatch(long courseId, Date date, Map<Long, String> statuses, Map<Long, String>
remarks,
                         long markedBy, String via) throws SQLException {
        String sql = "INSERT INTO attendance (course_id, student_id, att_date, status, remarks,
marked_by, marked_via) "
                   + "VALUES (?,?,?,?,?,?,?) "
                   + "ON DUPLICATE KEY UPDATE status = VALUES(status), remarks =
VALUES(remarks), "
                   + "marked_by = VALUES(marked_by), marked_via = VALUES(marked_via)";
        try (Connection c = DBUtil.getConnection()) {
            c.setAutoCommit(false);
            try (PreparedStatement ps = c.prepareStatement(sql)) {
                for (Map.Entry<Long, String> e : statuses.entrySet()) {
                    ps.setLong(1, courseId);
                    ps.setLong(2, e.getKey());
                    ps.setDate(3, date);
                    ps.setString(4, e.getValue());
                    String rem = remarks == null ? null : remarks.get(e.getKey());
                    if (rem == null || rem.isEmpty()) ps.setNull(5, Types.VARCHAR); else
ps.setString(5, rem);
                    if (markedBy > 0) ps.setLong(6, markedBy); else ps.setNull(6, Types.BIGINT);
                    ps.setString(7, via);
                    ps.addBatch();
                }
                ps.executeBatch();
                c.commit();
                return statuses.size();
            } catch (SQLException ex) {
                c.rollback();
                throw ex;
```

```
            }
        }
    }
```

6.7 Attendance Percentage Logic

The percentage is calculated in SQL by the view `v_attendance_summary` as `100 * SUM(status IN ('PRESENT','LATE')) / COUNT(*)`. A late student is counted as present, which is the rule used by most institutions; the rule can be changed in the view alone. The bean `AttendanceSummary` adds three helpers used by the pages: `isLow()`, `getLevel()` (good / ok / low for the colour of the bar) and `getClassesNeeded()`. If a student has attended *a* of *n* classes and the threshold is *t* percent, the number *x* of consecutive classes that must be attended to reach the threshold follows from (a + x) / (n + x) >= t / 100, that is x >= (t.n - 100.a) / (100 - t). For example, 19 of 30 classes with t = 75 gives x = (2250 - 1900) / 25 = 14 classes, exactly the value displayed on the student dashboard in Figure 8.12.

6.8 Servlets (Controllers)

`BaseServlet` defines `get()` and `post()` methods that may throw any exception, wraps them in `doGet` / `doPost` and forwards unexpected errors to the error page. The servlet that takes attendance first checks that the logged-in teacher owns the course, validates the date and that every enrolled student has a valid status, and only then calls the DAO:

Listing 6.7: TeacherAttendanceServlet.post()

```
    @Override
    protected void post(HttpServletRequest req, HttpServletResponse resp) throws Exception {
        Course c = ownCourse(req, resp, "courseId");
        if (c == null) return;
        LocalDate today = LocalDate.now();
        LocalDate date = WebUtil.parseDate(req.getParameter("date"), null);
        if (date == null || date.isAfter(today)) {
            WebUtil.flash(req, "error", "Choose a valid date (not in the future).");
            redirect(req, resp, "/teacher/attendance?courseId=" + c.getId());
            return;
        }
        Map<Long, String> statuses = new LinkedHashMap<>();
        Map<Long, String> remarks = new LinkedHashMap<>();
        for (StudentInfo s : courses.findEnrolled(c.getId())) {
            String st = WebUtil.trim(req.getParameter("status_" + s.getUserId()));
            if (!AttendanceDAO.isValidStatus(st)) {
                WebUtil.flash(req, "error", "Please mark every student before saving (missing: "
+ s.getFullName() + ").");
                redirect(req, resp, "/teacher/attendance?courseId=" + c.getId() + "&date=" +
date);
                return;
            }
            statuses.put(s.getUserId(), st);
            String rem = WebUtil.trim(req.getParameter("remark_" + s.getUserId()));
            remarks.put(s.getUserId(), rem.length() > 200 ? rem.substring(0, 200) : rem);
        }
        if (statuses.isEmpty()) {
            WebUtil.flash(req, "error", "No students are enrolled in this course yet.");
        } else {
            int n = attendance.saveBatch(c.getId(), WebUtil.sqlDate(date), statuses, remarks,
user(req).getId(), "WEB");
            log(req, "ATTENDANCE_SAVED", c.getCode() + " " + date + " students=" + n);
            WebUtil.flash(req, "success", "Attendance for " + c.getCode() + " on " + date + "
saved (" + n + " students).");
        }
        redirect(req, resp, "/teacher/attendance?courseId=" + c.getId() + "&date=" + date);
```

After a successful POST the servlet redirects to a GET (the *Post/Redirect/Get* pattern), so that refreshing the browser cannot submit the form twice. Messages are shown once through a *flash* attribute in the session.

6.9 JSP Views

The pages share `header.jspf` and `footer.jspf` through a static include; the header builds the navigation according to `sessionScope.user.role`. Progress bars cannot use inline `style` attributes because of the Content-Security-Policy, so the CSS file contains generated utility classes `w-0` ... `w-100` and the JSP writes `class="bar-fill low w-63"`. The same technique draws the trend chart on the admin dashboard.

Listing 6.8: Segmented P / L / A control for one student (teacher/attendance.jsp)

```
        <tr>
          <td><c:out value="${s.rollNo}"/></td>
          <td><c:out value="${s.fullName}"/></td>
          <td>
            <div class="seg">
              <input type="radio" id="p${s.userId}" name="status_${s.userId}" value="PRESENT" $
{cur == 'PRESENT' ? 'checked' : ''}><label class="seg-p" for="p${s.userId}">P</label>
              <input type="radio" id="l${s.userId}" name="status_${s.userId}" value="LATE" ${cur
== 'LATE' ? 'checked' : ''}><label class="seg-l" for="l${s.userId}">L</label>
              <input type="radio" id="a${s.userId}" name="status_${s.userId}" value="ABSENT" $
{cur == 'ABSENT' ? 'checked' : ''}><label class="seg-a" for="a${s.userId}">A</label>
            </div>
          </td>
          <td><input class="remark" name="remark_${s.userId}" maxlength="200" value="<c:out
value='${existing[s.userId].remarks}'/>" placeholder="e.g. medical leave"></td>
        </tr>
      </c:forEach>
      </tbody>
    </table>
    </div>
```

6.10 REST API and API Keys

`ApiServlet` is mapped to `/api/v1/*` and routes by path: `GET /health`, `/courses`, `/courses/{code}/students`, `/attendance`, `/students/{roll}/summary` and `POST /attendance/mark`. Responses use the envelope `{"success": true, "data": ...}`. A small hand-written `JsonUtil` parses and writes JSON so that no external library is required. The mark endpoint accepts either one scan or a bulk list, validates every record (roll number exists, status valid, student enrolled) and reports rejected records individually, returning 200 when all were saved, 207 for partial success and 422 when nothing was saved.

An API key has the form `sams_` followed by 32 random bytes in URL-safe Base64. When the administrator presses *Generate*, the key is displayed once; only its SHA-256 hash and the first twelve characters are stored. If a database backup leaks, no key can be recovered from it. Keys can be revoked, re-activated or deleted, and the *last used* time is updated on every call.

Listing 6.9: Bulk attendance call and its partial-success response

```
curl -X POST http://localhost:8080/AttendanceManagementSystem/api/v1/attendance/
mark \
     -H "X-API-KEY: sams_xxxxxxxxxxxxxxx" -H "Content-Type: application/json" \
     -d '{"course":"CS502","date":"2026-11-05",
          "records":[{"roll":"CS2026003","status":"LATE"},
                     {"roll":"NOPE","status":"PRESENT"}]}'

HTTP/1.1 207
{"success":true,"data":{"course":"CS502","date":"2026-11-05","saved":1,
  "rejected":[{"roll":"NOPE","reason":"unknown roll number"}]}}
```

6.11 Audit Log

`AuditDAO.log()` is called for logins (successful and failed), logouts, password changes, user, course and enrolment changes, attendance saves, API key operations, CSV exports and API

marking. A failure to write the log never breaks the main action. The administrator sees the latest 25 events on the Reports page (Figure 8.8).

6.12 Front-End Assets

`style.css` defines colours as CSS variables, a flexible-box layout, cards, tiles, tables, badges, progress bars, the segmented control and a print style sheet that hides navigation when a report is printed. `app.js` is wrapped in an immediately invoked function and handles: confirmation dialogs (`data-confirm`), auto-submitting selects (`data-autosubmit`), live table search (`data-filter`), showing student-only fields, 'mark all' buttons with live counters, copy-to-clipboard for new API keys, and a print button.

CHAPTER 7
TESTING

7.1 Testing Strategy

Testing followed the levels recommended by Sommerville [10] and Pressman [11]. **Unit-level checks** were made while writing each DAO and utility class (for example, verifying the PBKDF2 round trip and the percentage formula against hand calculations). **Integration and system testing** was automated with a shell script, `tests/smoke_test.sh`, which drives the real running application over HTTP with `curl`, logs in as each role, submits real forms with real CSRF tokens, calls the REST API and inspects the database with SQL queries. **Usability checks** were carried out by opening every page in a browser at desktop and phone width.

7.2 Test Environment

Table 7.1: Test environment

Item	Value
Server	Apache Tomcat 9 (embedded launcher) on Ubuntu 24.04, Java 21
Database	MariaDB 10.11 with the demo data of `sample_data.sql`
JDBC driver	MariaDB Connector/J 2.7.6
Test client	curl 8, Headless Chromium for screenshots
Data	25 users, 4 courses, 80 enrolments, 2,400 attendance rows

7.3 Test Cases and Results

Table 7.2: Executed test cases (all passed)

ID	Area	Test	Expected result / result
TC-01	Authentication	Open /login	HTTP 200 - **Pass**
TC-02	Authentication	POST /login without CSRF token	HTTP 403 - **Pass**
TC-03	Authentication	Login with wrong password	Stays on login page with error - **Pass**
TC-04	Authentication	Login as admin with correct password	302 redirect to dashboard - **Pass**
TC-05	Access control	Open /admin/dashboard without session	302 redirect to login - **Pass**
TC-06	Access control	Admin opens /teacher/dashboard	HTTP 403 - **Pass**
TC-07	Admin	GET dashboard, users, courses, reports, API keys, enrolments (6 pages)	HTTP 200 each - **Pass**
TC-08	Admin	Create student 'student99' with valid data	Row in users and students - **Pass**
TC-09	Data security	Inspect stored password of the new user	Starts with 'pbkdf2$', not clear text - **Pass**
TC-10	Authentication	Login as teacher1	302 redirect - **Pass**
TC-11	Teacher	Open dashboard, attendance, report and CSV export of own course (4 pages)	HTTP 200 each - **Pass**
TC-12	Access control	Teacher opens a course of another teacher	HTTP 403 - **Pass**
TC-13	Access control	Teacher opens /admin/users	HTTP 403 - **Pass**
TC-14	Teacher	Save attendance for 20 students (1 absent)	302; 20 rows for today in DB - **Pass**
TC-15	Teacher	Save the same date again	Still exactly 20 rows (no duplicates) - **Pass**
TC-16	Teacher	Check the absent student's record	Status = ABSENT - **Pass**
TC-17	Teacher	Download CSV export	Header line correct, UTF-8 BOM present - **Pass**

ID	Area	Test	Expected result / result
TC-18	Authentication	Login as student01	302 redirect - **Pass**
TC-19	Student	Open dashboard and own history (2 pages)	HTTP 200 each - **Pass**
TC-20	Access control	Student opens /teacher/dashboard	HTTP 403 - **Pass**
TC-21	Student	Dashboard lists own course CS501	Course code present - **Pass**
TC-22	API	GET /api/v1/health without key	HTTP 200 - **Pass**
TC-23	API	GET /api/v1/courses without key	HTTP 401 - **Pass**
TC-24	API	GET /api/v1/courses with invalid key	HTTP 401 - **Pass**
TC-25	API	Admin generates a key; GET /courses with it	HTTP 200 - **Pass**
TC-26	API	GET /students/CS2026001/summary	JSON contains CS501 - **Pass**
TC-27	API	POST single scan (status in lower case)	HTTP 200 - **Pass**
TC-28	API	Check the stored row	marked_via = API - **Pass**
TC-29	API	POST malformed JSON	HTTP 400 - **Pass**
TC-30	API	POST a date in the future	HTTP 400 - **Pass**
TC-31	API	Admin revokes the key; call again	HTTP 401 - **Pass**

Some rows of Table 7.2 contain several checks (for example TC-07 opens six pages), so the script executed **41 individual assertions in total, and all 41 passed**. Figure 7.1 shows the assertions per module. In addition, a bulk call with one valid and one unknown roll number returned HTTP 207 and listed the rejected record, as designed.

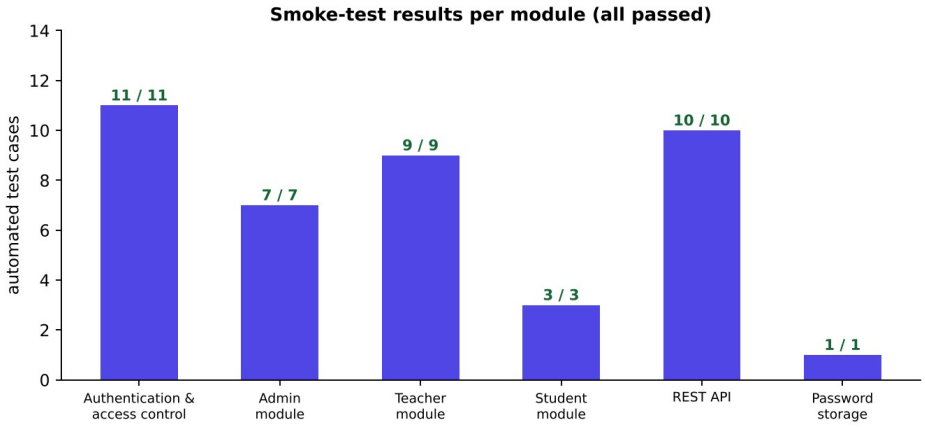


Figure 7.1: Automated assertions per module

7.4 Defects Found and Fixed

Table 7.3: Defects found during testing

No.	Defect	Cause	Fix
1	Dates on the dashboard chart and history page appeared as 2026-11-04 instead of 04 Nov 2026, although the pattern was specified.	The JSTL `fmt:formatDate` tag falls back to `toString()` when no formatting locale is available (requests without an Accept-Language header).	Added the context parameter `javax.servlet.jsp.jstl.fmt.locale = en` to `web.xml`; verified on all pages.
2	Early version allowed progress bar widths only through inline style.	The Content-Security-Policy blocks inline styles, so bars would not render.	Generated CSS classes `w-0`..`w-100` and `h-0`..`h-100`; JSP writes the class name.

7.5 Performance Measurement

Response times were measured with 60 consecutive requests per URL (the first five discarded) against the demo database of 2,400 attendance rows, on the same machine as the server. The figures include the start-up of the `curl` process itself and therefore slightly overestimate the server time.

Table 7.4: Measured response time

URL	Average (ms)	Median (ms)	95th percentile (ms)
Login page	6.7	6.4	9.8
Admin dashboard (4 aggregate queries + chart)	38.7	37.6	54.2
Admin reports (defaulters + audit log)	33.3	35.2	47.9
API: list courses	14.9	14.7	21.9
API: attendance search (200 rows)	11.8	10.0	21.5

All measured values satisfy the non-functional requirement of 100 ms for pages and 50 ms for API calls. The whole database, including indexes, occupies about 864 KB.

CHAPTER 8
USER MANUAL WITH SCREENSHOTS

This chapter walks through the running system. All screenshots were taken from the real application with the demo data; none of them is a mock-up.

8.1 Login Page

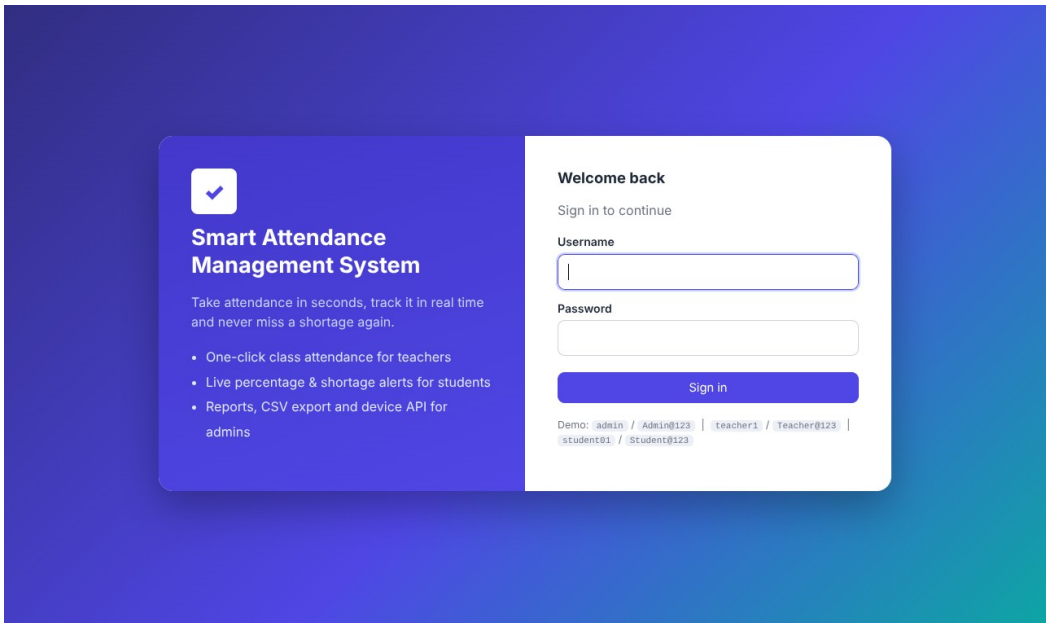


Figure 8.1: Login page

Every user starts at the login page. The left panel describes the system; the right panel contains the sign-in form with a hidden CSRF token. For the demo, the credentials of one user per role are printed below the form (remove this line in production).

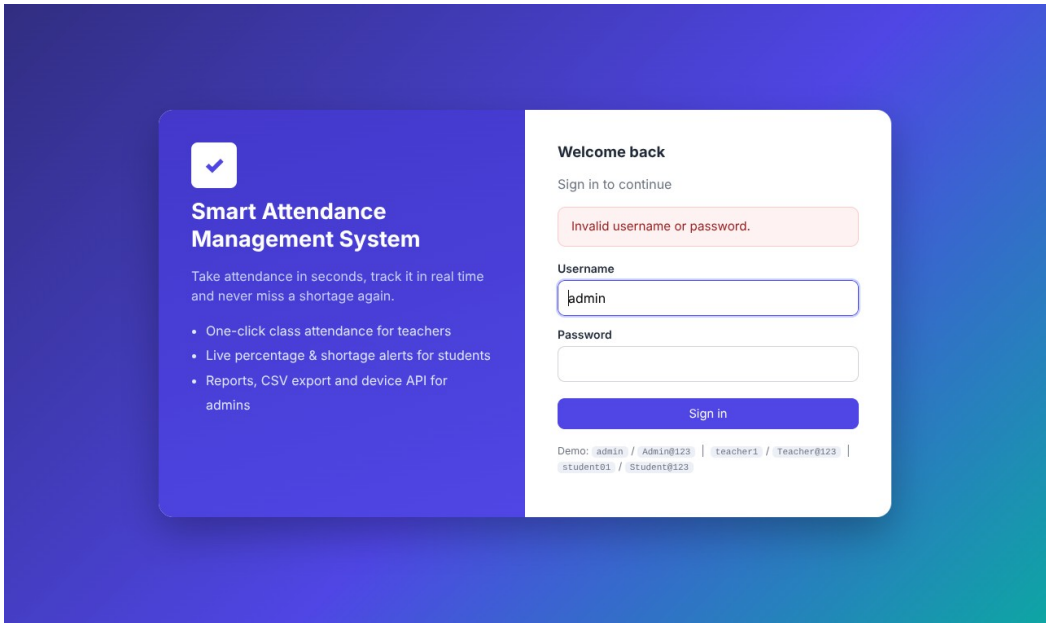


Figure 8.2: Login failure message

A wrong password produces a neutral message that does not reveal whether the username exists. After five failed attempts the pair username + IP address is locked for five minutes.

8.2 Administrator Portal

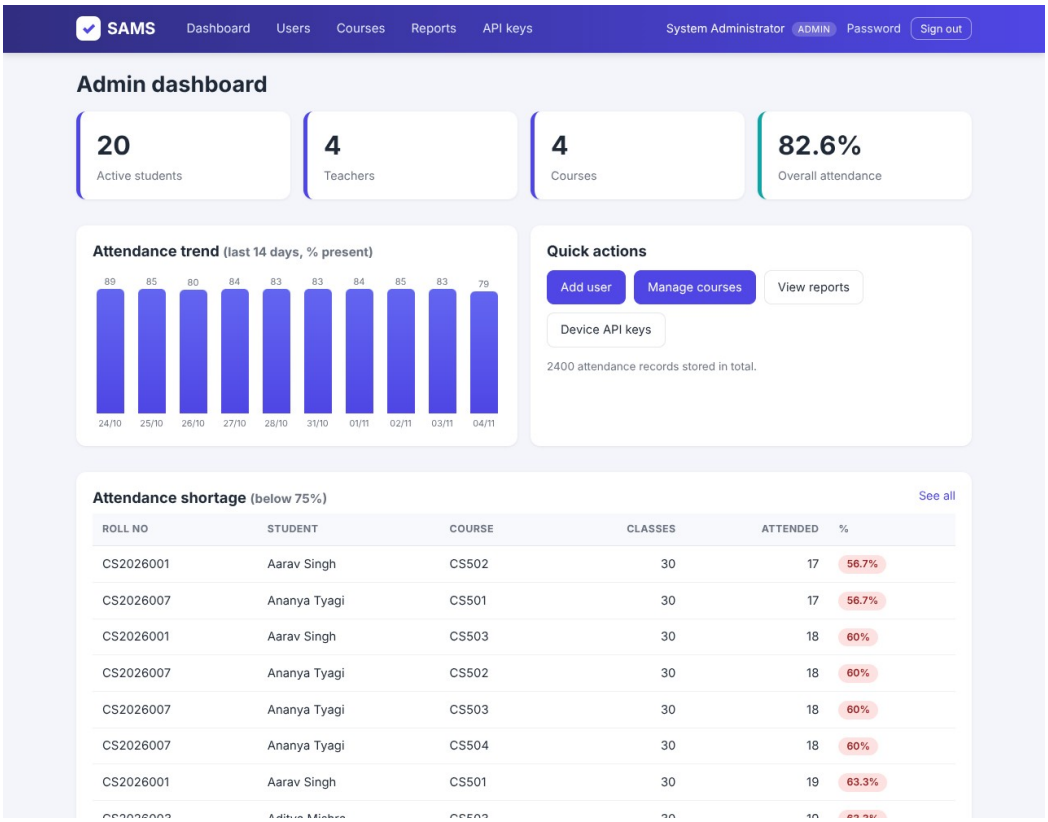


Figure 8.3: Admin dashboard

The dashboard shows the number of active students, teachers and courses, the overall attendance percentage, a 14-day trend chart (each column is the percentage present on that day) and the first eight defaulters with the percentage in a red badge.

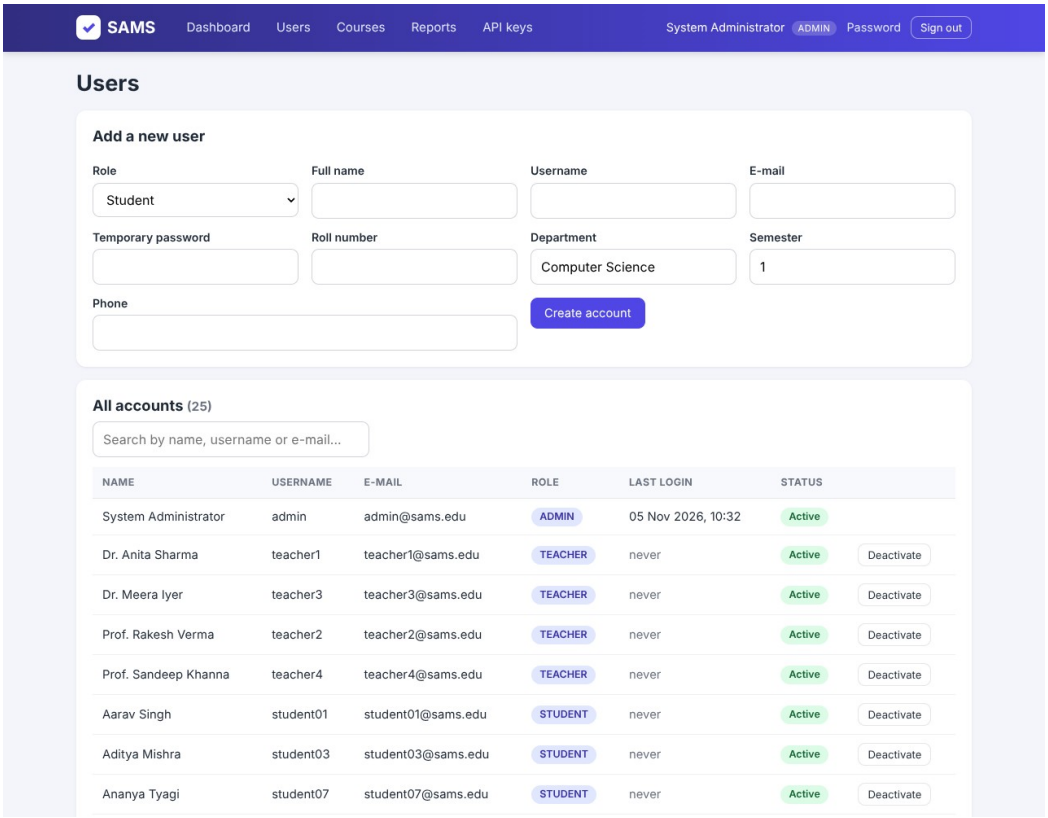


Figure 8.4: User management

The top form creates a user. Selecting the role *Student* shows the extra fields roll number, department, semester and phone. Below it, all accounts are listed with a live search box, last-login time and an Activate / Deactivate button. An administrator cannot deactivate himself.

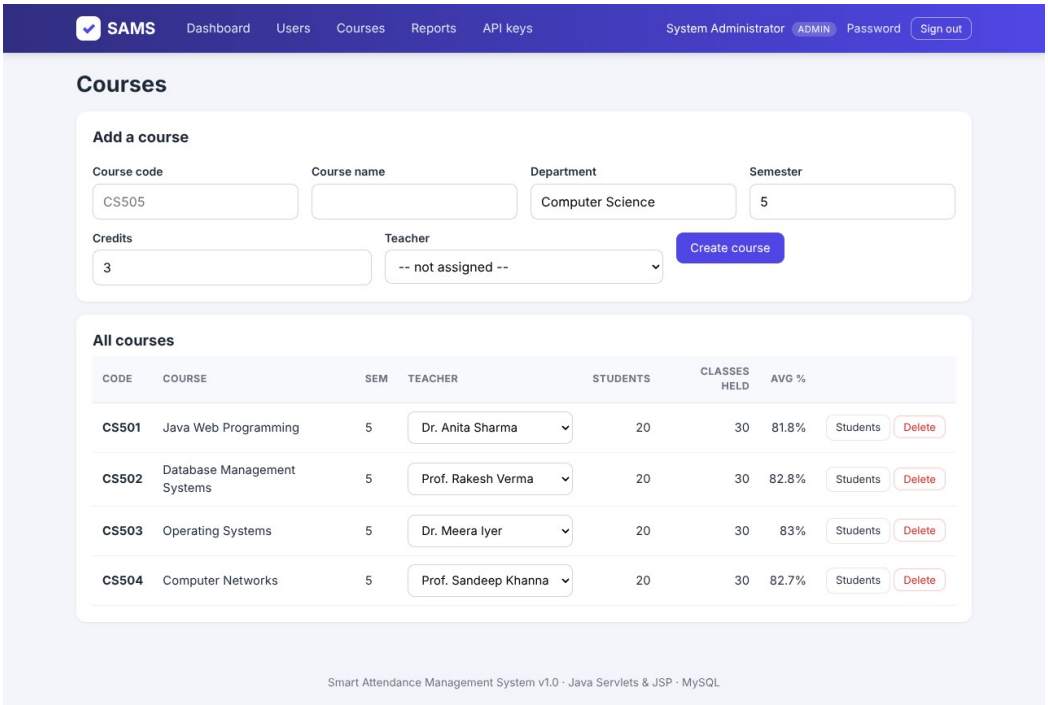


Figure 8.5: Course management

Courses are added with code, name, department, semester, credits and teacher. The table shows enrolled students, number of classes held and average attendance; the teacher can be changed with the drop-down (it submits automatically).

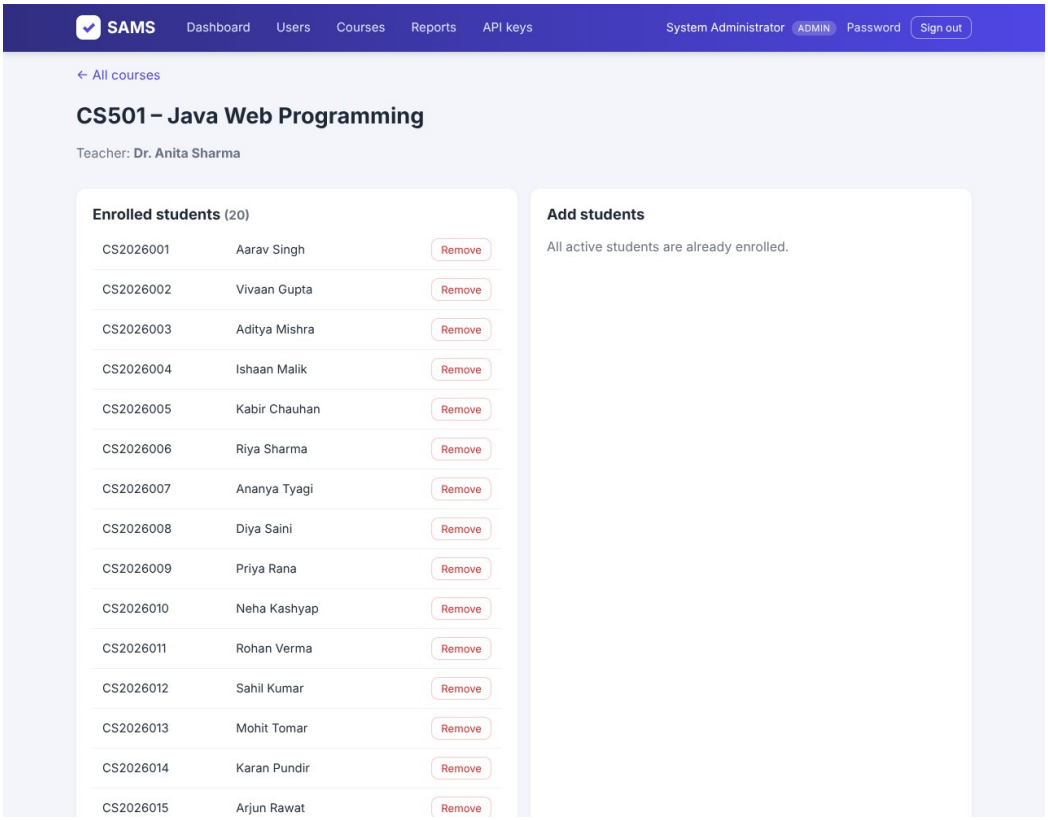


Figure 8.6: Enrolment of students

The left card lists students already in the course, with a Remove button; the right card lists active students that are not enrolled, with check boxes for bulk enrolment.

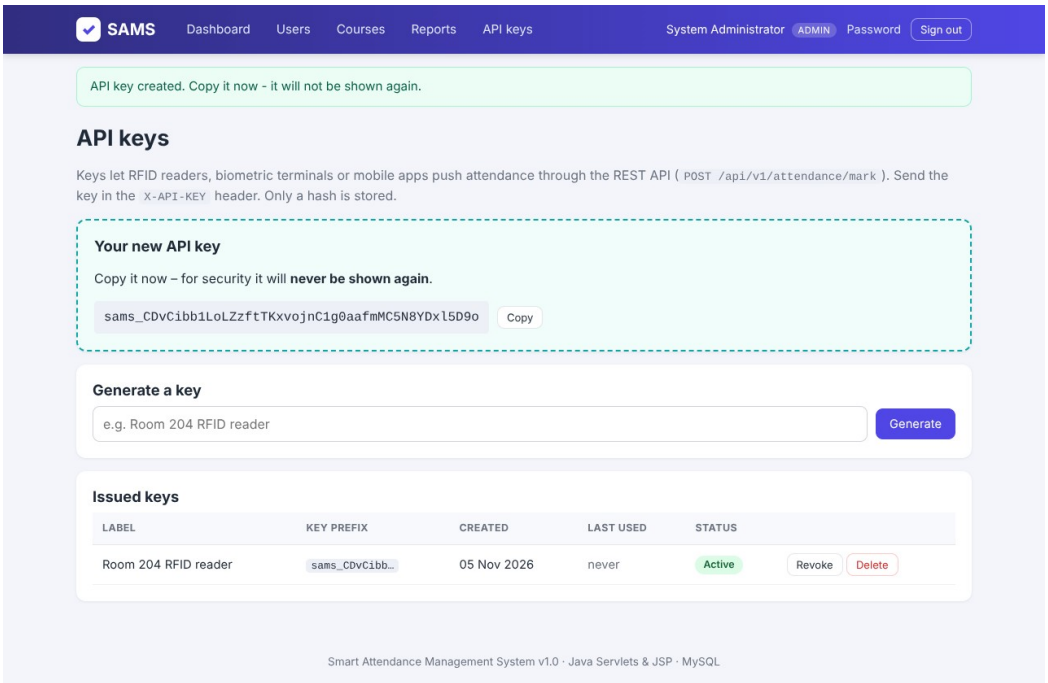


Figure 8.7: API key management

A new key is displayed once in a highlighted box with a Copy button. The table afterwards shows only the label and the first twelve characters. Keys can be revoked, re-activated or deleted.

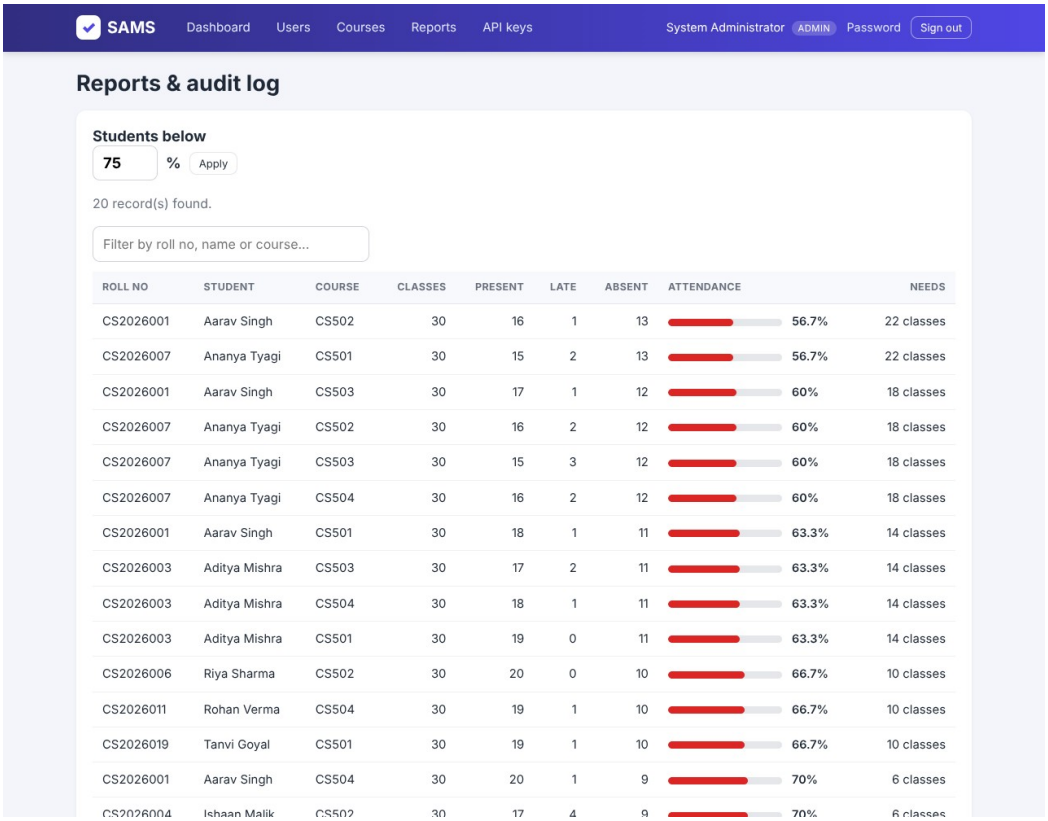


Figure 8.8: Reports and audit log

The defaulter report lists every student-course pair below a chosen percentage with present, late and absent counts, a progress bar and the number of classes needed to recover. The audit log below records who did what, when and from which IP address.

8.3 Teacher Portal

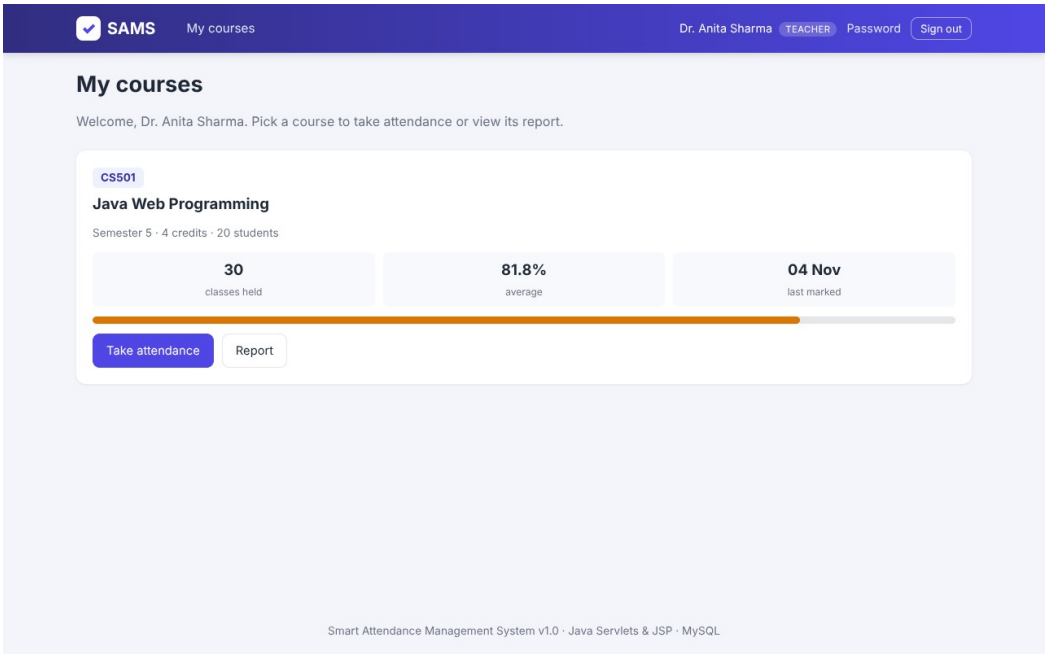


Figure 8.9: Teacher dashboard

Each course assigned to the teacher appears as a card with classes held, average attendance and date of the last marking. The buttons open the attendance page or the report.

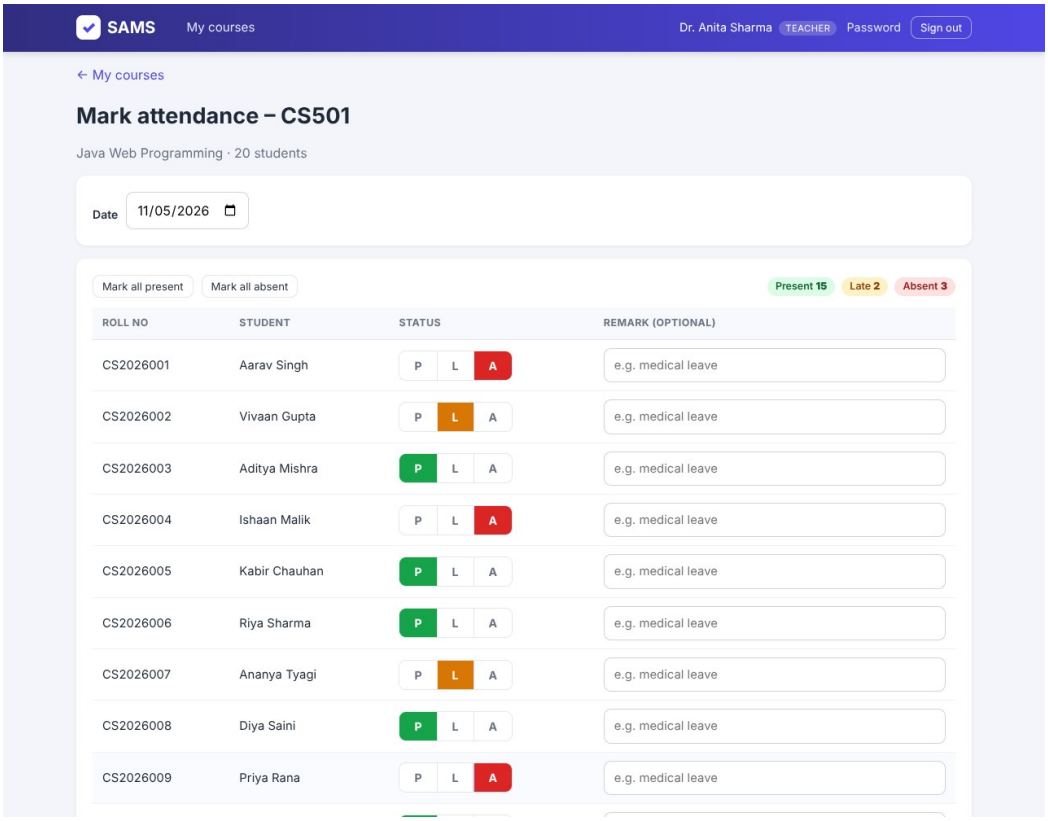


Figure 8.10: Taking attendance

The date defaults to today; changing it re-opens that day with the saved marks, which is how mistakes are corrected. Every student is Present by default; the teacher only touches the exceptions (Late or Absent) and may add a remark. The counters at the top right update live. *Save attendance* stores all rows in one transaction.

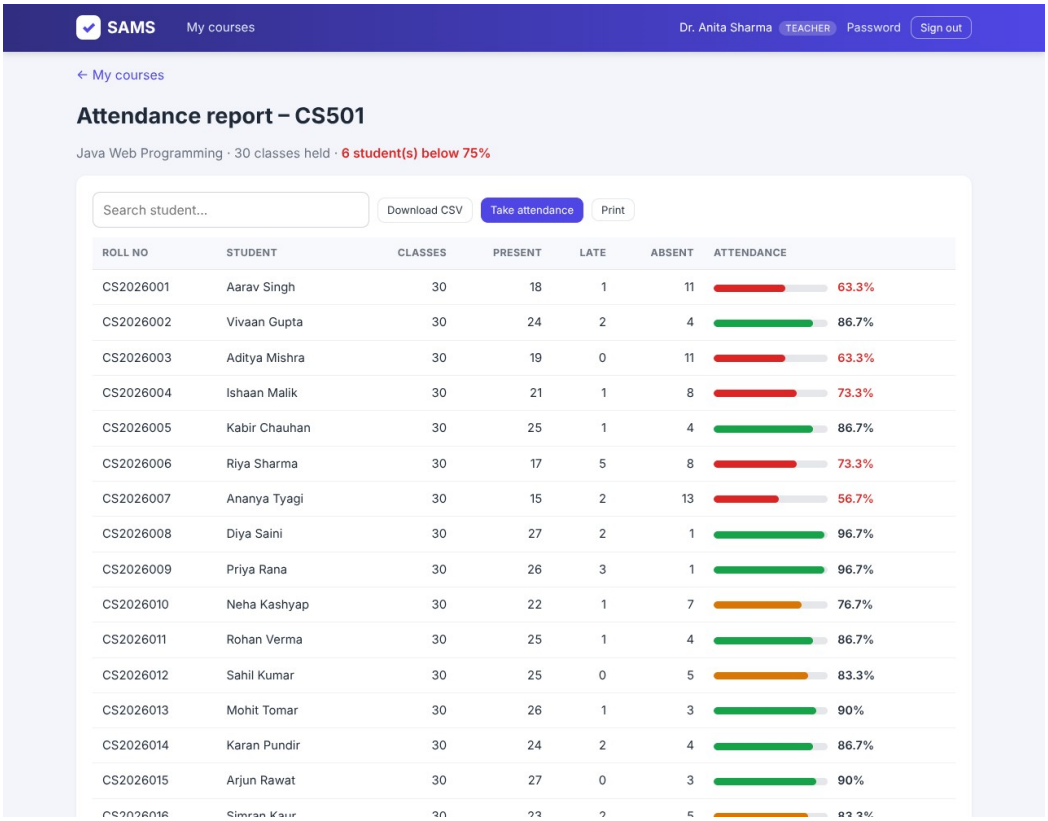


Figure 8.11: Course report

The report lists classes, present, late and absent counts and a colour-coded percentage for each student. It can be searched, printed or downloaded as a CSV file that opens correctly in Excel.

8.4 Student Portal

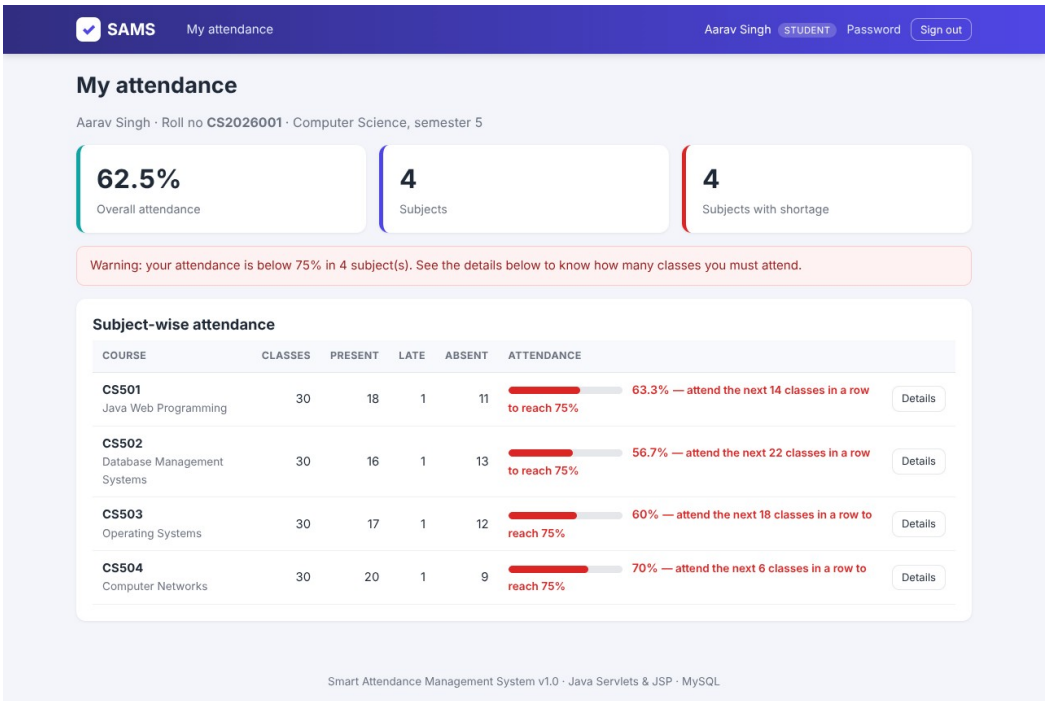


Figure 8.12: Student dashboard

The student sees the overall percentage, the number of subjects and the number of subjects with shortage. Below 75 percent the row turns red and says how many classes in a row must be attended to reach the threshold. Here, 14 classes are needed in CS501 (19 of 30 attended).

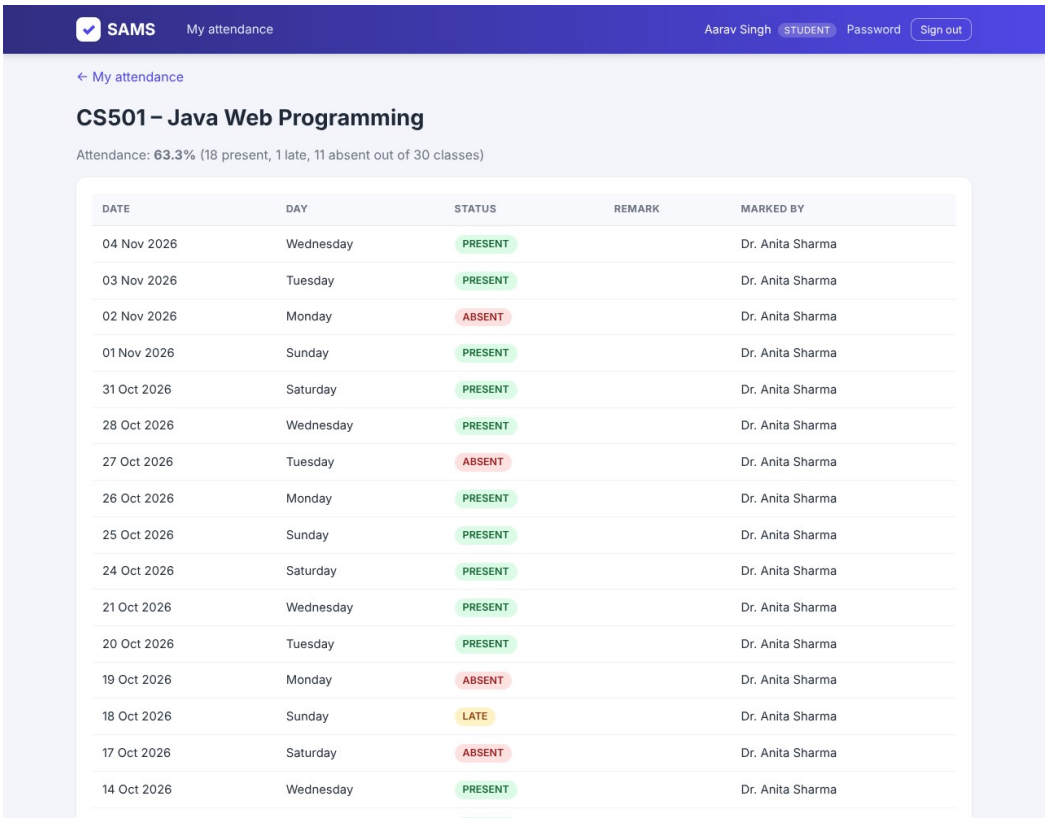


Figure 8.13: Date-wise history

The *Details* button shows every class date with the day of the week, the status, the teacher's remark and who marked it (a teacher's name or *Device (API)*).

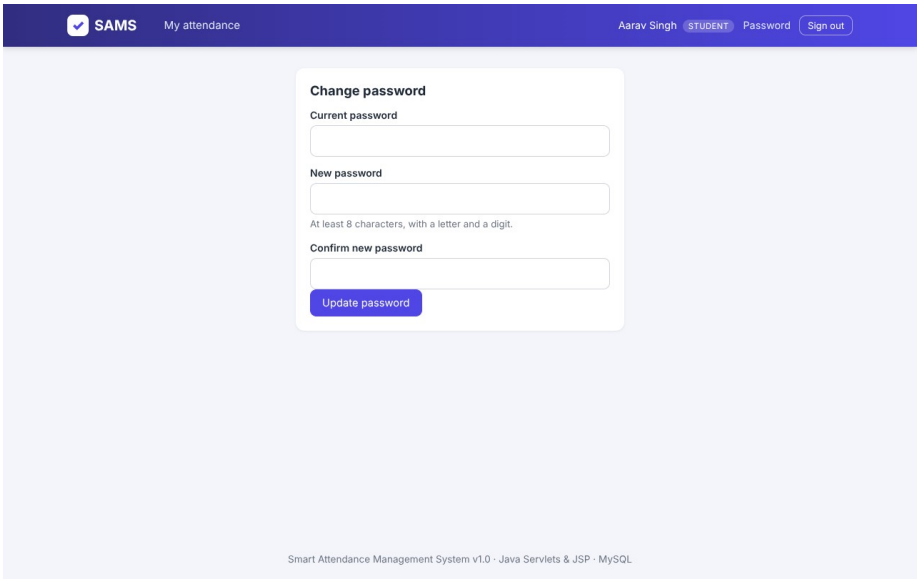


Figure 8.14: Change password page

Every user can change the password. The system checks the current password and enforces at least eight characters with a letter and a digit.

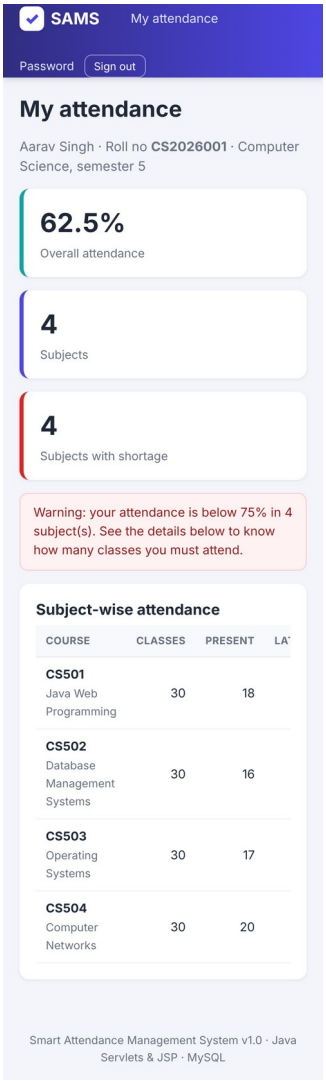


Figure 8.15: Student dashboard on a phone (390 px wide)

The layout adapts to narrow screens: boxes wrap, tables scroll horizontally inside their own container and the user name is hidden to save room.

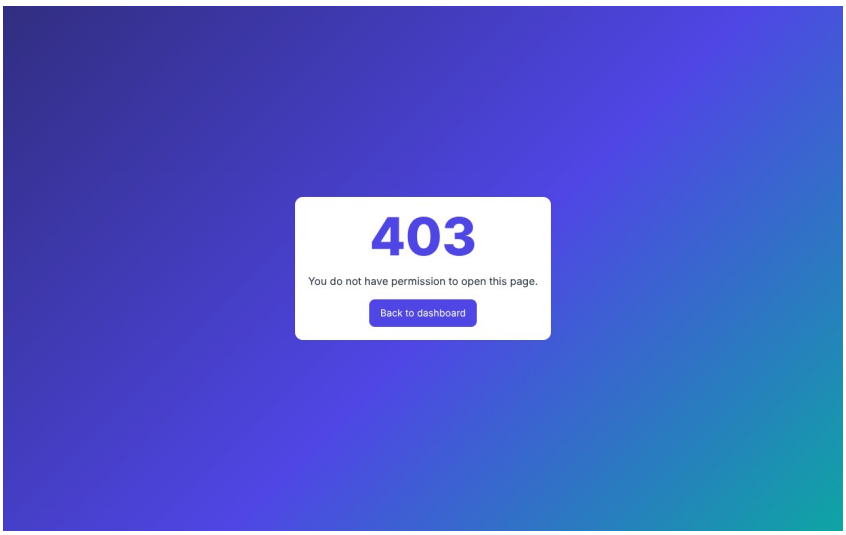


Figure 8.16: Access-denied page

If an admin types a teacher URL the role filter answers with HTTP 403 and a friendly page instead of exposing any data.

CHAPTER 9
DEPLOYMENT GUIDE

9.1 Overview

The delivery package contains both the **source project** (Maven) and a **prebuilt WAR** in the folder `dist`. The prebuilt WAR contains the MariaDB JDBC driver and works out of the box with XAMPP / MariaDB; the Maven build uses MySQL Connector/J for MySQL 8.

9.2 Step-by-Step Installation

1. Install a JDK or JRE 11 or later and check with `java -version`.
2. Install Apache Tomcat 9.0.x (unzip it, for example to `C:\tomcat9`).
3. Install MySQL 8 or XAMPP (MariaDB) and start the database server.
4. Create the database: `mysql -u root -p < database/schema.sql`, then optionally load the demo data with `mysql -u root -p < database/sample_data.sql`. The script also creates the user `sams_user`.
5. Copy `dist/AttendanceManagementSystem.war` (or the WAR built by `mvn clean package`) to `<TOMCAT_HOME>/webapps`.
6. Start Tomcat with `bin/startup.bat` (Windows) or `bin/startup.sh` (Linux / macOS).
7. Open `http://localhost:8080/AttendanceManagementSystem/` and sign in with the demo administrator account.
8. Change all demo passwords and delete the demo users that are not needed.

9.3 Configuration

Table 9.1: Configuration keys (db.properties or conf/sams.properties)

Key	Default	Meaning
db.driver	com.mysql.cj.jdbc.Driver	JDBC driver class (org.mariadb.jdbc.Driver in the prebuilt WAR)
db.url	jdbc:mysql://localhost:3306/ sams_db?...	JDBC URL of the database
db.user / db.password	sams_user / Sams@2026	Database account - change in production
db.pool.size	10	Maximum simultaneous connections
attendance.threshold	75	Minimum percentage; students below it are flagged

To keep the secrets outside the WAR, create the file `<TOMCAT_HOME>/conf/sams.properties` containing only the keys that must change, for example `db.password=MyStrongPassword`.

9.4 Production Recommendations

- Run Tomcat behind HTTPS (a reverse proxy such as Nginx or Apache httpd, or Tomcat's own TLS connector) and set the `Secure` flag on the session cookie.
- Create a database account with only the rights needed on `sams_db`, and use a strong password.
- Back up daily: `mysqldump -u root -p sams_db > backup.sql`.
- Give every RFID reader or app its own API key so that one key can be revoked without affecting the others.

- Remove the demo hint line from `login.jsp` and the demo data from the database.

9.5 Troubleshooting

Table 9.2: Common problems

Symptom	Likely cause and remedy
HTTP 404 on the home page	The URL must contain the WAR name; check `logs/catalina.out` for deployment errors.
`JDBC driver not found`	Rebuild with Maven so the driver is in `WEB-INF/lib`, or copy the driver JAR to `<TOMCAT_HOME>/lib`.
`Access denied for user 'sams_user'`	Run `schema.sql` again or correct the user and password in the configuration.
Server time-zone error	Set `serverTimezone=` in the JDBC URL.
Blank page / `javax` vs `jakarta` errors	Use Tomcat 9.x; Tomcat 10 and later need the `jakarta.*` namespace.
Dates look like 2026-11-04	Check that the `javax.servlet.jsp.jstl.fmt.locale` parameter is present in `web.xml`.

CHAPTER 10

RESULTS AND DISCUSSION

10.1 Objectives Versus Achievement

Table 10.1: Achievement of the objectives

Objective	Result
1. Java Servlet / JSP / JDBC application with MySQL	Achieved: 41 classes, 16 JSP files, 7 tables and 1 view.
2. Mark a class in under a minute, correct old days	Achieved: default-present design with 'mark all'; editing a past date updates records, no duplicates (TC-14, TC-15).
3. Automatic percentage and shortage warning	Achieved: SQL view + 'classes needed' calculation on the student dashboard.
4. Administrator tools	Achieved: users, courses, enrolments, defaulters, audit log, API keys.
5. REST API with API keys	Achieved: 6 endpoints, hashed keys, revocation, rate limit (TC-22..TC-31).
6. Security	Achieved: PBKDF2, CSRF, roles, lock-out, headers, audit log (Chapter 5.9).
7. Responsive interface	Achieved: tested at 1280 px and 390 px width.
8. Testing and documentation	Achieved: 41 / 41 automated assertions passed; this report.

10.2 Analysis of the Demo Data

The demo database was generated with deliberately different attendance habits per student. It contains 80 student-course pairs (20 students x 4 courses) with 30 class days each, i.e. 2,400 attendance records. The overall attendance is 82.6 percent, and 20 of the 80 pairs are below the 75 percent threshold, which are exactly the cases the dashboards are designed to expose (Figure 10.1).

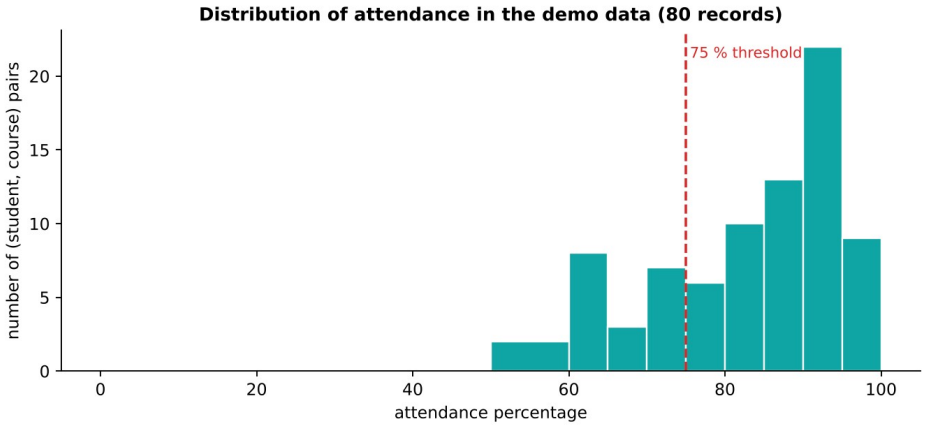


Figure 10.1: Distribution of attendance percentages

10.3 Benefits Observed

- **Time:** a 20-student class is marked in a few clicks because only exceptions have to be touched.
- **Accuracy:** the percentage is computed by the database from the records; no manual counting is involved.
- **Transparency:** students see their record at any time, and every change by staff is logged.

- **Automation:** an RFID reader can write attendance through one HTTP call with an API key.
- **Low cost:** the whole system runs on free software on one ordinary computer.

10.4 Comparison with the Manual Process

Table 10.2: Manual process versus SAMS

Task	Manual	SAMS
Mark a class of 60	3 - 5 minutes	about 1 minute (mostly exceptions)
Monthly percentage report for a class	30 - 60 minutes of counting	instant, one click
Find all defaulters of a department	hours	instant (Reports page)
Student checks own record	ask the teacher	24 x 7 on the dashboard
Detect a changed entry	impossible	audit log

CHAPTER 11

LIMITATIONS AND FUTURE SCOPE

11.1 Limitations

- Statuses are limited to Present, Late and Absent; there is no separate *excused leave* status.
- Every class day is one record per course; several lectures of the same course on one day (periods) are not distinguished.
- No e-mail or SMS notification is sent; students must open the dashboard to see warnings.
- The application has to be deployed behind HTTPS by the administrator; TLS is not built in.
- The user interface is in English only.
- Attendance by face recognition or QR code needs external software that calls the API.

11.2 Future Enhancements

1. **Notifications:** daily e-mail / SMS / WhatsApp message to students and parents when the percentage falls below the threshold.
2. **Timetable and periods:** support for several periods per day and automatic class lists from a timetable.
3. **Leave management:** leave applications with teacher approval that mark the days as *Excused*.
4. **QR-code attendance:** the teacher shows a rotating QR code that students scan with the phone; the scan calls the API.
5. **Face recognition module:** a camera service that recognises students and posts attendance through the existing API.
6. **Analytics:** weekly trend per student, early-warning prediction of defaulters, department comparisons and PDF reports.
7. **Mobile application and multi-language interface.**
8. **Multi-institution mode** with a separate database schema per institution.
9. **Single sign-on** (LDAP / OAuth) and two-factor authentication for administrators.

CHAPTER 12
CONCLUSION

The Smart Attendance Management System demonstrates that the everyday problem of attendance can be solved completely with standard, free Java web technology. The application replaces the paper register by a central database, gives teachers a fast way to mark and correct attendance, gives students an always-available view of their percentage with an exact recovery target, and gives administrators dashboards, defaulter lists and an audit trail.

The project applied the main ideas of enterprise Java in one coherent system: the MVC architecture with servlets and JSP, the DAO pattern over JDBC, transactions, filters for cross-cutting security, a hand-built connection pool, hashed passwords, CSRF protection, role-based access control and a documented REST API secured with hashed API keys. The system was verified with 41 automated end-to-end assertions, all of which passed, and the measured response times are far below the targets set in the requirements.

Beyond the working software, the project gave practical experience in requirement analysis, UML and data-flow modelling, relational database design, secure coding, testing and technical documentation. The architecture is open for extension; notifications, QR-code or face-recognition attendance and analytics can be added on top of the existing layers and API without redesigning the core.

REFERENCES

[1] Oracle Corporation, Java Servlet Specification, Version 4.0 (JSR 369), 2017.

[2] Oracle Corporation, JavaServer Pages Specification, Version 2.3, 2013.

[3] Apache Software Foundation, Apache Tomcat 9 Documentation, https://tomcat.apache.org/tomcat-9.0-doc/.

[4] Oracle Corporation, Java Database Connectivity (JDBC) API Documentation, https://docs.oracle.com/javase/.

[5] Oracle Corporation, MySQL 8.0 Reference Manual, https://dev.mysql.com/doc/.

[6] K. Moriarty, B. Kaliski, A. Rusch, PKCS #5: Password-Based Cryptography Specification Version 2.1, RFC 8018, IETF, 2017.

[7] OWASP Foundation, Cross-Site Request Forgery Prevention Cheat Sheet, https://cheatsheetseries.owasp.org/.

[8] OWASP Foundation, Password Storage Cheat Sheet, https://cheatsheetseries.owasp.org/.

[9] M. Fowler, Patterns of Enterprise Application Architecture, Addison-Wesley, 2002.

[10] I. Sommerville, Software Engineering, 10th ed., Pearson, 2015.

[11] R. S. Pressman, B. R. Maxim, Software Engineering: A Practitioner's Approach, 8th ed., McGraw-Hill, 2014.

[12] R. T. Fielding, Architectural Styles and the Design of Network-based Software Architectures, Doctoral dissertation, University of California, Irvine, 2000.

[13] IEEE, IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998.

Appendix A: Database Script (schema.sql)

```
-- ======================================================================
--  Smart Attendance Management System (SAMS)  -  Database schema
--  Works on MySQL 8.x and MariaDB 10.x
--  Run:  mysql -u root -p < database/schema.sql
-- ======================================================================
DROP DATABASE IF EXISTS sams_db;
CREATE DATABASE sams_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE sams_db;

-- ----------------------------------------------------------------------
-- 1. USERS : every person who can log in (admin / teacher / student)
-- ----------------------------------------------------------------------
CREATE TABLE users (
    id             BIGINT       NOT NULL AUTO_INCREMENT,
    username       VARCHAR(50)  NOT NULL,
    password_hash  VARCHAR(255) NOT NULL,
    full_name      VARCHAR(100) NOT NULL,
    email          VARCHAR(120) NOT NULL,
    role           ENUM('ADMIN','TEACHER','STUDENT') NOT NULL,
    active         TINYINT(1)   NOT NULL DEFAULT 1,
    created_at     TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    last_login     TIMESTAMP    NULL,
    PRIMARY KEY (id),
    UNIQUE KEY uq_users_username (username),
    UNIQUE KEY uq_users_email (email),
    KEY idx_users_role (role)
) ENGINE=InnoDB;

-- ----------------------------------------------------------------------
-- 2. STUDENTS : extra profile data for users whose role = STUDENT
-- ----------------------------------------------------------------------
CREATE TABLE students (
    user_id     BIGINT      NOT NULL,
    roll_no     VARCHAR(30) NOT NULL,
    department  VARCHAR(60) NOT NULL,
    semester    TINYINT     NOT NULL,
    phone       VARCHAR(20) NULL,
    PRIMARY KEY (user_id),
    UNIQUE KEY uq_students_roll (roll_no),
    CONSTRAINT fk_students_user FOREIGN KEY (user_id)
        REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- ----------------------------------------------------------------------
-- 3. COURSES : subjects taught by a teacher
-- ----------------------------------------------------------------------
CREATE TABLE courses (
    id           BIGINT       NOT NULL AUTO_INCREMENT,
    course_code  VARCHAR(20)  NOT NULL,
    course_name  VARCHAR(120) NOT NULL,
    department   VARCHAR(60)  NOT NULL,
    semester     TINYINT      NOT NULL,
    credits      TINYINT      NOT NULL DEFAULT 3,
    teacher_id   BIGINT       NULL,
    created_at   TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_courses_code (course_code),
    KEY idx_courses_teacher (teacher_id),
    CONSTRAINT fk_courses_teacher FOREIGN KEY (teacher_id)
        REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB;

-- ----------------------------------------------------------------------
-- 4. ENROLLMENTS : which student studies which course (many-to-many)
-- ----------------------------------------------------------------------
CREATE TABLE enrollments (
    id           BIGINT NOT NULL AUTO_INCREMENT,
    course_id    BIGINT NOT NULL,
    student_id   BIGINT NOT NULL,
    enrolled_on  DATE   NOT NULL DEFAULT (CURRENT_DATE),
    PRIMARY KEY (id),
    UNIQUE KEY uq_enrollment (course_id, student_id),
    CONSTRAINT fk_enr_course  FOREIGN KEY (course_id)  REFERENCES courses(id) ON DELETE CASCADE,
    CONSTRAINT fk_enr_student FOREIGN KEY (student_id) REFERENCES users(id)   ON DELETE CASCADE
) ENGINE=InnoDB;

-- ----------------------------------------------------------------------
-- 5. ATTENDANCE : one row per student, per course, per day
--    The UNIQUE key makes double marking impossible and lets us use
--    INSERT ... ON DUPLICATE KEY UPDATE to correct a mistake safely.
-- ----------------------------------------------------------------------
```

```
CREATE TABLE attendance (
    id          BIGINT      NOT NULL AUTO_INCREMENT,
    course_id   BIGINT      NOT NULL,
    student_id  BIGINT      NOT NULL,
    att_date    DATE        NOT NULL,
    status      ENUM('PRESENT','ABSENT','LATE') NOT NULL,
    remarks     VARCHAR(200) NULL,
    marked_by   BIGINT      NULL,
    marked_via  ENUM('WEB','API') NOT NULL DEFAULT 'WEB',
    marked_at   TIMESTAMP   NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_attendance (course_id, student_id, att_date),
    KEY idx_att_student (student_id),
    KEY idx_att_date (att_date),
    CONSTRAINT fk_att_course  FOREIGN KEY (course_id)  REFERENCES courses(id) ON DELETE CASCADE,
    CONSTRAINT fk_att_student FOREIGN KEY (student_id) REFERENCES users(id)   ON DELETE CASCADE,
    CONSTRAINT fk_att_marker  FOREIGN KEY (marked_by)  REFERENCES users(id)   ON DELETE SET NULL
) ENGINE=InnoDB;

-- ---------------------------------------------------------------------
-- 6. API_KEYS : keys handed to external devices (RFID / biometric / apps)
--    Only the SHA-256 hash of a key is stored, never the key itself.
-- ---------------------------------------------------------------------
CREATE TABLE api_keys (
    id           BIGINT       NOT NULL AUTO_INCREMENT,
    label        VARCHAR(80)  NOT NULL,
    key_prefix   VARCHAR(12)  NOT NULL,
    key_hash     CHAR(64)     NOT NULL,
    active       TINYINT(1)   NOT NULL DEFAULT 1,
    created_by   BIGINT       NULL,
    created_at   TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    last_used_at TIMESTAMP    NULL,
    PRIMARY KEY (id),
    UNIQUE KEY uq_api_key_hash (key_hash),
    CONSTRAINT fk_key_user FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB;

-- ---------------------------------------------------------------------
-- 7. AUDIT_LOG : who did what and when (security & accountability)
-- ---------------------------------------------------------------------
CREATE TABLE audit_log (
    id          BIGINT       NOT NULL AUTO_INCREMENT,
    user_id     BIGINT       NULL,
    action      VARCHAR(50)  NOT NULL,
    details     VARCHAR(400) NULL,
    ip_address  VARCHAR(45)  NULL,
    created_at  TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    KEY idx_audit_time (created_at)
) ENGINE=InnoDB;

-- ---------------------------------------------------------------------
-- Report view : attendance percentage per student per course
-- ---------------------------------------------------------------------
CREATE OR REPLACE VIEW v_attendance_summary AS
SELECT  c.id            AS course_id,
        c.course_code,
        c.course_name,
        u.id            AS student_id,
        s.roll_no,
        u.full_name     AS student_name,
        COUNT(a.id)                                          AS total_classes,
        SUM(a.status = 'PRESENT')                            AS present_count,
        SUM(a.status = 'LATE')                               AS late_count,
        SUM(a.status = 'ABSENT')                             AS absent_count,
        ROUND(100 * SUM(a.status IN ('PRESENT','LATE')) / NULLIF(COUNT(a.id),0), 1) AS percentage
FROM enrollments e
JOIN courses  c ON c.id = e.course_id
JOIN users    u ON u.id = e.student_id
JOIN students s ON s.user_id = u.id
LEFT JOIN attendance a ON a.course_id = e.course_id AND a.student_id = e.student_id
GROUP BY c.id, c.course_code, c.course_name, u.id, s.roll_no, u.full_name;

-- ---------------------------------------------------------------------
-- Application database user (used by src/main/resources/db.properties)
-- ---------------------------------------------------------------------
CREATE USER IF NOT EXISTS 'sams_user'@'localhost' IDENTIFIED BY 'Sams@2026';
GRANT ALL PRIVILEGES ON sams_db.* TO 'sams_user'@'localhost';
FLUSH PRIVILEGES;
```